Adapting a Six-Metric XAI Evaluation Framework to a Construction-Safety Vision-Language Model: A Fourteen-Day Pilot

Prepared for research collaboration with Professor Mustafa Abdallah

June 23, 2026

Contents

Chapter 1

Establishing a Reproducible GPU Environment for Florence-2 (Day 1)

1.1 Motivation

The first day of the pilot was scoped narrowly and deliberately: confirm that the pilot is executable at all before any evaluation work begins. Concretely, this meant demonstrating that (i) the software environment loads and recognizes the GPU, (ii) a real sample from the target dataset can be retrieved, and (iii) the target vision-language model can run a single inference pass on that sample and return a usable output. Until these three preconditions were verified directly, every subsequent day's plan — sampling, masking, attention extraction, stability and robustness testing — would have rested on an unverified assumption. Day 1's purpose was to remove that assumption.

1.2 Implementation

The pilot codebase was scaffolded as an installable Python package, `src/xai_pilot` (commit `c30c4a9`), rather than as a loose collection of scripts. Two modules carried the environment-level logic established on Day 1: `config.py`, which centralizes shared paths and identifiers, and `model.py`, which encapsulates Florence-2 loading and task execution.

`config.py` fixes three values that recur throughout the rest of the pilot: `HF_DATASET_ID = "LouisChen15/ConstructionSite"`, `MODEL_ID = "microsoft/Florence-2-base-ft"`, and `SEED = 42`. Pinning these in one place, rather than inlining them per script, was intended to keep every later day's sampling and inference deterministic and traceable back to a single source of truth.

`model.py` is responsible for loading Florence-2 in a way that actually succeeds on the target (Windows) platform. Florence-2's custom modeling code is loaded via `trust_remote_code=True` and, as shipped, unconditionally imports `flash_attn`. Because no usable Windows wheel exists for `flash_attn`, `model.py` monkeypatches `transformers.dynamic_module_utils.get_imports` to drop `flash_attn` from the list of required imports before `from_pretrained` executes, and separately forces `attn_implementation="sdpa"` so that the model is never asked to use the unavailable attention backend. Loading is otherwise the standard Hugging Face pattern: `AutoModelForCausalLM.from_pretrained(model_id, trust_remote_code=True, ...)`.

The verification step itself was implemented as `scripts/01_check_environment.py`. This script loads the real ConstructionSite test split in streaming mode, loads Florence-2-base-ft, runs the `<CAPTION>` task on one real sample drawn from that split, and exits with a non-zero status if CUDA is unavailable, if the dataset fails to load, or if the model fails to load — so that a successful run is itself evidence the environment is sound, rather than a claim that has to be taken on faith.

1.3 Architecture

Figure 1.1 summarizes the pipeline as it existed at the end of Day 1: a streaming read from the ConstructionSite dataset on the Hugging Face Hub, a single forward pass through Florence-2-base-ft running in fp16 on CUDA with the `sdpa` attention implementation, the `<CAPTION>` task token, and a caption plus an accompanying confidence proxy as output. No masking, attention extraction, sampling-based stability checks, or perturbation logic exist yet at this stage; those are introduced in later chapters and, where relevant, the present pipeline is extended rather than replaced.

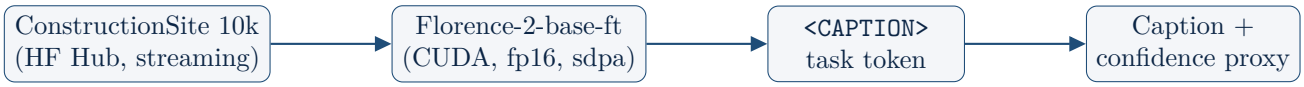


Figure 1.1: Pipeline architecture after Day 1: environment and single-image inference only.

1.4 Worked Example

The output below was not copied from Day 1's original log. It is a fresh execution of `scripts/01_check_environment.` reproduced verbatim during the preparation of this report, in the same spirit as the from-scratch reproducibility rerun performed on Day 13. This is stated explicitly because the distinction matters: the numbers that follow are evidence that the Day 1 environment remains reproducible months later, not merely a record of what once happened.

```
torch 2.12.1+cu130, cuda available:  True
Sample image_id=0000001, size=(1200, 675)
Model device:  cuda:0, dtype:  torch.float16
Caption:  A large truck with a crane on the back of it.
Mean token probability (confidence proxy):  0.6442
Inference time:  1565.3 ms
```

The run confirms all three preconditions identified in Section 1.1 in a single execution: CUDA is available and a CUDA device is in use; a real, correctly-shaped sample (size 1200×675) was retrieved from the dataset's test split; and Florence-2 produced a coherent caption together with a mean-token-probability confidence proxy and an inference time, on a single image, without manual intervention.

1.5 Problems Encountered and Resolutions

Three concrete problems were encountered in establishing this environment, each requiring a deliberate resolution rather than a default installation path.

1. `flash_attn` import crash on Windows. Florence-2's remote modeling code unconditionally imports `flash_attn` on load, and no usable Windows wheel for `flash_attn` exists. *Resolution*: `model.py` monkeypatches `transformers.dynamic_module_utils.get_imports` to remove `flash_attn` from the list of imports the loader requires, and explicitly forces `attn_implementation="sdpa"` in the call to `from_pretrained`, so the model never attempts to use the unavailable backend.

2. A `transformers>=5.0` config-load crash. `transformers` versions at or above 5.0 removed legacy generation attributes (for example, `forced_bos_token_id`) from `PretrainedConfig`. Florence-2's pinned remote-code `configuration_florence2.py` still expects these attributes to exist, and crashes

on load under `transformers>=5.0`. *Resolution*: `requirements.txt` pins `transformers==4.49.0`, the last 4.x release, avoiding the breaking change entirely rather than attempting to patch around it.

3. A silent CPU-only `torch` install. A plain `pip install torch`, or allowing `torch` to be pulled in as a side effect of installing the rest of the pilot's dependencies, silently installs a CPU-only wheel, with no error to signal that GPU support is missing. *Resolution*: `requirements.txt` documents that `torch` must be installed first and separately, pinned to its exact CUDA build (`torch==2.12.1+cu130`), from the matching CUDA index, before any other dependency is installed.

1.6 Standard vs. Non-Standard Choices

It is worth separating, explicitly, which parts of this setup follow documented practice and which do not. The *standard* choice is the loading pattern itself: `AutoModelForCausalLM.from_pretrained(..., trust_remote_code=True)` is a documented Hugging Face mechanism for loading models that ship custom modeling code, and Florence-2 is used through that mechanism as intended.

Two choices, by contrast, are *non-standard*. The `flash_attn` import patch is a Windows-specific workaround with no upstream documentation; it relies on monkeypatching an internal `transformers` utility function rather than any officially supported configuration option, and may need to be revisited if that internal function's interface changes in a future `transformers` release. The hard pin to `transformers==4.49.0` is similarly a workaround rather than a preference: it was forced by an upstream breaking change in `transformers>=5.0`, not chosen for any feature in 4.49.0 itself, and the pilot's reproducibility is therefore contingent on that specific upstream version continuing to be installable.

1.7 Implications for the Paper

Florence-2-base-ft, the vision-language model used throughout this pilot, has 231.6M parameters and is released under the MIT license [8]; the dataset used to source sample images is ConstructionSite 10k, hosted on the Hugging Face Hub [6]. Both facts are reported here because the model size, license, and dataset provenance are exactly the kind of detail that determines whether a result is reproducible by a third party without access to this codebase.

More generally, the three problems documented in this chapter — a platform-specific import failure, a breaking upstream dependency change, and a silent hardware-backend misconfiguration — are the kind of detail that is easy to omit from a methods section and expensive to omit in practice: any of the three, left unresolved, would have produced either an outright crash or a silently CPU-bound, effectively unusable pipeline, without changing any of the pilot's downstream numbers in a way that would be obvious after the fact. We recommend that the eventual paper report the software environment as a methods footnote or appendix — minimally, the exact `transformers` and `torch` versions, the attention implementation forced at load time, and the platform-specific workaround required to load the model at all — rather than relying on a requirements file alone to carry that information. With the environment, dataset access, and a single forward pass confirmed working, the pilot's next step is to move from one manually inspected image to a defined, reproducible sample of the dataset suitable for systematic evaluation.

Chapter 2

Constructing a Balanced, Rule-Mapped Pilot Sample (Day 2)

2.1 Motivation

With the environment established in Chapter 1, the pilot needed a fixed, finite sample to evaluate against, rather than an open-ended stream of dataset rows. ConstructionSite 10k's test split contains thousands of images covering four safety conditions of interest — PPE violations, fall hazards, struck-by risk, and full compliance — in proportions set by the dataset itself, not by the pilot's evaluation needs. Drawing a plain random sample from that split risks a sample dominated by whichever class happens to be most common, leaving the rarer classes too thin to support any later per-class analysis. Day 2's task was therefore to construct a sample that is (i) balanced across the four classes the pilot cares about, (ii) explicitly mapped to the specific safety rule each sample is meant to test, and (iii) reproducible from a fixed seed, before any inference or evaluation work began.

2.2 Implementation

Three functions, added to `data.py` in commit `c76a642`, carry Day 2's logic, together with a new `RULE_QUERIES` table in `prompts.py` and a driver script, `scripts/02_select_samples.py`, that writes the resulting sample to `pilot_samples.csv`.

`classify_image(row)` maps a dataset row's rule-violation fields to one of four classes. A row may violate zero, one, or several of the four safety rules; `classify_image` returns a single `primary_class`, taken as the highest-priority violated class according to `config.py`'s `CLASS_PRIORITY = ["ppe_violation", "fall_hazard", "struck_by_risk", "compliant"]`. A row that violates both rule 1 and rule 3, for example, is classified `ppe_violation` rather than `fall_hazard`, because `ppe_violation` sits earlier in the priority list. Separately, `violated_rule_ids` records every rule the row actually violates, not only the one that determined `primary_class`.

`assign_rule_id` then picks the one rule each sample is tested against in Days 3–10. `config.py`'s `RULE_TO_CLASS` maps `rule_1_violation` to `ppe_violation`, `rule_2_violation` and `rule_3_violation` both to `fall_hazard`, and `rule_4_violation` to `struck_by_risk`. For a violation sample, the assigned rule is the one unambiguous violated rule that maps to its `primary_class`. Compliant samples have no ground-truth rule to assign, since by definition they violate none; these are instead round-robined evenly across `COMPLIANT_ROUND_ROBIN = ["rule_1", "rule_2", "rule_3", "rule_4"]`. This round-robin is a deliberate design choice: it gives balanced "model correctly says compliant" coverage for each rule, rather than testing the same rule against every compliant image and leaving the other three rules with no compliant counterexamples at all.

`select_balanced_sample` performs the actual selection in a single streaming pass over the dataset. For each of the four classes, a reservoir of candidate samples is kept, capped at `n_per_class * 10`

(500 candidates) to bound memory while streaming. Once reservoirs are populated, the final sample is a seeded (`SEED = 42`) random draw of `n_per_class` (`SAMPLES_PER_CLASS = 50`) from each class's reservoir, making the selection reproducible from the same seed and dataset stream.

`prompts.py` additionally introduces `RULE_QUERIES`, two phrasings per presence-based rule used later to query Florence-2: `rule_1: ["hard hat", "high-visibility vest"]`, `rule_2: ["safety harness", "fall-protection lanyard"]`, and `rule_3: ["guardrail", "edge protection barrier"]`. `PERSON_PHRASE = "worker"` and `RULE_4_PROXIMITY_PAIR = ("worker", "excavator")` support rule 4's proximity-based check, which is structurally different from the presence checks used for rules 1–3. `RULE_QUERIES` is not yet consumed on Day 2; it is written here so that Day 3's inference step has a single, already-defined source of query phrasings to draw from.

2.3 Results

`scripts/02_select_samples.py` streamed the ConstructionSite test split once and wrote the selected sample to `pilot_samples.csv`. The result is 163 total rows, not the 200 originally planned (50 per class across four classes):

Class	Count
`ppe_violation`	50
`fall_hazard`	50
`compliant`	50
`struck_by_risk`	13
Total	163

Three of the four classes filled to the planned target of 50. `struck_by_risk` filled to only 13, capped by genuine scarcity in the underlying dataset rather than by any defect in the sampling procedure: the entire 3,004-image test split contains only 13 `struck_by_risk` examples in total, a fact confirmed directly by an exhaustive scan of the full split rather than assumed from the reservoir's behavior.

2.4 Problems Encountered and Resolutions

The shortfall in `struck_by_risk` samples was the one substantive problem encountered on Day 2. It surfaced as `select_balanced_sample`'s reservoir for that class filling to only 13 candidates regardless of how much of the stream was consumed. Rather than treat this as a sampling bug to be patched, it was investigated directly: a full scan of the test split confirmed that 13 is the true population count for `struck_by_risk` in that split, not an artifact of the streaming or reservoir logic. *Resolution*: proceed with `n=13` for `struck_by_risk` rather than block the pilot on a class the dataset cannot supply at the planned size. This shortfall is flagged explicitly as a limitation here, and it recurs in every later chapter's `struck_by_risk` numbers.

2.5 Standard vs. Non-Standard Choices

The sampling strategy is, in its broad outline, standard practice: stratified, class-balanced sampling implemented via per-class reservoir sampling with a fixed seed, so the same sample can be reconstructed

from the same dataset stream. This part of the design follows well-established sampling methodology and is not particular to this pilot.

The round-robin assignment of rules to compliant samples, by contrast, is non-standard: it is a domain-specific balancing choice, motivated by the absence of a natural ground-truth rule for compliant images, and is not a pattern found in generic sampling libraries. It was introduced here specifically so that each of the four safety rules receives roughly equal compliant-sample coverage for later "correctly identifies compliance" evaluation, rather than leaving that coverage to chance.

2.6 Implications for the Paper

The 13-vs-50 class imbalance in `struck_by_risk` is a property of the ConstructionSite 10k dataset itself, not a limitation of the sampling method used to draw from it, and should be reported as such in any paper's Dataset or Limitations section. Because the imbalance was confirmed by an exhaustive scan rather than inferred from the pilot's own reservoir behavior, it can be stated with confidence rather than hedged. This scarcity directly motivates an open question raised again in Chapter 14: whether a future iteration of this work should scale up data collection specifically for underrepresented classes such as `struck_by_risk`, or instead accept and explicitly correct for the imbalance during evaluation. With a fixed, balanced, rule-mapped sample of 163 images now in hand, the pilot's next step is to run baseline inference over that sample.

Chapter 3

Baseline Inference and the Discovery of a Sensitivity Failure: From Scene-Level to Person-Relative Grounding (Day 3)

3.1 Motivation

With a fixed, balanced, rule-mapped sample of 163 images in hand from Day 2, the pilot's next step was to produce a baseline judgment for every sample: does the model say each image complies with the safety rule it was assigned, or not? Every later evaluation day — masking-based Descriptive Accuracy, attention-based explanation extraction, Robustness, Bounded Completeness — depends on this baseline answer being a meaningful judgment in the first place, one that actually discriminates between compliant and violating images rather than defaulting to the same answer regardless of input. Day 3's task was therefore not only to run inference over the sample, but to verify, quantitatively, that the resulting proxy was fit for that purpose before any explanation method was built on top of it. As this chapter documents, that verification surfaced a serious problem in the most natural implementation of the proxy, and resolving it became the day's central piece of work.

3.2 Implementation

Florence-2-base-ft has no native free-form visual-question-answering task token: its task vocabulary consists of grounding and captioning tokens — `<CAPTION>`, `<OD>`, `<OPEN_VOCABULARY_DETECTION>`, `<CAPTION_TO_PHRASE_GROUNDING>` — rather than a head designed to answer arbitrary yes/no questions [8]. Each of the four safety rules therefore had to be reframed as a grounding query rather than posed as a direct question. `inference.py`'s `answer_rule()` dispatches on the rule being tested: rule 4 (struck-by risk, worker-to-excavator proximity) is routed to `_answer_rule_4`, which performs a proximity check between two independently grounded objects, `"worker"` and `"excavator"`; rules 1–3 (hard hat, harness, guardrail) are routed to `_answer_presence_rule`, which — in its first implementation — simply asks whether the relevant safety object can be grounded anywhere in the image at all.

`scripts/03_run_baseline_inference.py` ran this logic over all 163 samples from Day 2's `pilot_samples.csv`, writing one row per sample to `results/baseline_predictions.csv` and rendering 20 overlay images, with detected boxes drawn on the original frame, to `results/figures/baseline/`.

3.3 Architecture Before and After

Figure 3.1 contrasts the two versions of the rules 1–3 proxy built and run on Day 3. The left-hand side, v1, is the scene-level existence check described above: a single `<OPEN_VOCABULARY_DETECTION>` call for the safety object, and an answer that depends only on whether that call returns any box at all. The right-hand side, v2, is the person-relative redesign developed later the same day in response to the

failure documented in Section 3.6: two independent detection calls, one for `"worker"` and one for the safety object, followed by a per-worker coverage check rather than a single scene-wide existence check. Both versions share the same rule 4 proximity logic, which is not shown again here because it was not changed.

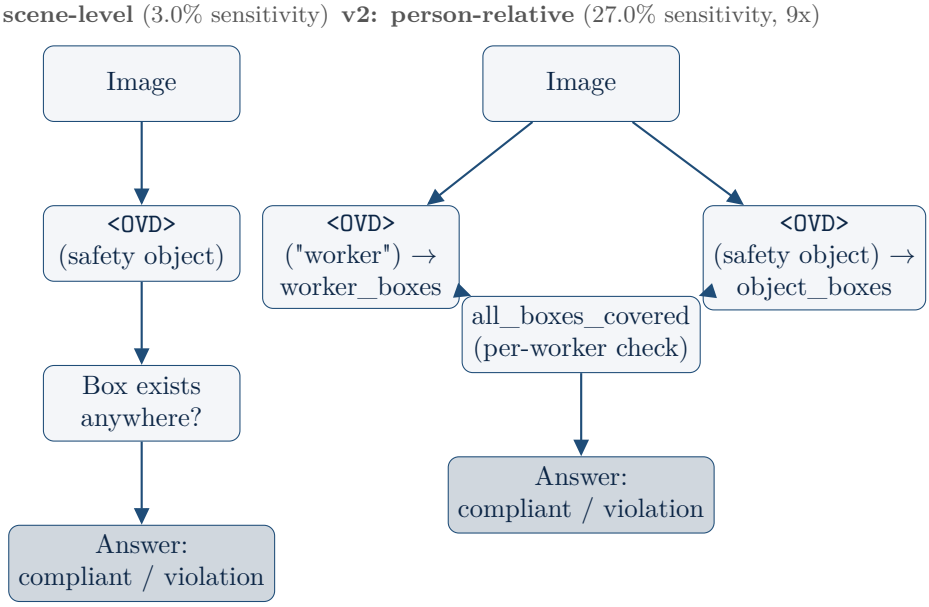


Figure 3.1: Safety-rule proxy logic before (left) and after (right) the Day 3 redesign.

3.4 Worked Examples

Before quantifying the proxy's accuracy, the pipeline's basic mechanics were validated by visual inspection of the rendered overlays. Figure 3.2 shows image 7: a rule 1 (hard hat) query, where `<OPEN_VOCABULARY_DETECTION>` returns a plausible, tightly-localized box landing correctly on a worker's head. The scene itself contains five workers, a detail that becomes important in Section 3.8's discussion of the residual single-worker-box limitation. Figure 3.3 shows image 120: rule 4's proximity check, where both `"worker"` and `"excavator"` are independently grounded, and the resulting boxes stand in a spatially sound relationship to one another. Together, these two examples confirm that the underlying detections are real and well-localized, not noise — a fact that turns out to matter directly in Section 3.6, where the v1 proxy's failure is traced to how the detections are *used*, not to any defect in the detections themselves.

Pipeline mechanics held up at the full sample size, not only on these two hand-picked examples: all 163 target `image_id`s from Day 2's sample were found in the dataset stream, and `assigned_rule_id` round-tripped correctly for every row. Mean inference time was 174ms per call and mean confidence (mean token probability) was 0.58, both consistent with Day 1's single-sample numbers and indicating that nothing changed about the model's basic operating characteristics when moving from one image to a 163-image batch.

3.5 Results

Figure 3.2: Image 7, rule 1 (hard hat): `<OPEN_VOCABULARY_DETECTION>` correctly localizes a hard hat on a worker's head. Five workers are visible in the original scene.

3.5.1 v1: scene-level proxy

With the pipeline confirmed mechanically sound, v1's predictions were compared against ground-truth `primary_class` to measure sensitivity (the fraction of true violations the proxy correctly flags as violations) and specificity (the fraction of true compliant images the proxy correctly flags as compliant). The presence-based proxy covering rules 1–3 and the proximity-based proxy covering rule 4 were scored separately, since they are structurally different checks.

Proxy (v1)	**Sensitivity**	**Specificity**
Presence-based (rules 1–3)	3.0% (3/100)	97.4% (37/38)
Proximity-based (rule 4)	61.5% (8/13)	41.7% (5/12)

The contrast between these two rows is stark and is the central finding of Day 3. The presence-based proxy achieves high specificity but almost no sensitivity at all: of 100 true violations across rules 1–3, it correctly flags only 3. A proxy with 3.0% sensitivity is, for practical purposes, a near-constant "compliant" classifier — it agrees with the dataset on compliant images almost every time, and disagrees on violations almost every time, which is the signature of a classifier that is not discriminating on the relevant signal at all. The proximity-based rule 4 proxy, by contrast, already shows real, balanced discriminative signal at this small sample size (62% sensitivity, 42% specificity on $n = 13$ and $n = 12$ respectively) and required no redesign.

Figure 3.3: Image 120, rule 4 (struck-by risk): the proximity check grounds both `"worker"` and `"excavator"` with a sound spatial relationship between the two boxes.

3.5.2 v2: person-relative proxy

After the redesign described in Section 3.6, `scripts/03_run_baseline_inference.py` was re-run on all 163 samples with the updated `_answer_presence_rule` logic. The results:

Proxy (v2)	**Sensitivity**	**Specificity**
Presence-based (rules 1–3), overall	**27.0% (27/100)**	73.7% (28/38)
rule_1 (hard hat)	16.0% (8/50)	—
rule_2 (harness)	30.8% (4/13)	—
rule_3 (guardrail)	40.5% (15/37)	—
Proximity-based (rule 4), unchanged	61.5% (8/13)	41.7% (5/12)

Overall presence-based sensitivity rises from 3.0% to **27.0%** — a **9x improvement** — at the cost of specificity falling from 97.4% to 73.7%. This is an expected, and in this case a desirable, precision/recall tradeoff: the v1 proxy's high specificity was an artifact of a classifier that almost never says "violation," not evidence of genuine discriminative skill, and the v2 proxy's lower specificity is the necessary consequence of a proxy that has started to actually discriminate instead of defaulting to "compliant." Per-rule sensitivity is uneven: rule 3 (guardrail) benefits most, rising to 40.5%, plausibly because its looser proximity threshold (described below) is closer in spirit to the relational rule 4 check that already

worked well; rule 1 (hard hat) improves least, to 16.0%, though even this is more than five times its v1 value.

3.6 Problems Encountered and Resolutions

The 3.0% sensitivity figure for the v1 presence-based proxy was the one substantial problem encountered on Day 3, and diagnosing its root cause was the day's central piece of analytical work.

Root cause. This was not a code bug: the underlying detections are real, well-localized boxes, a fact already confirmed visually in Section 3.4's worked examples. The failure is a mismatch between what the dataset's ground-truth label means and what the v1 query actually asks. The dataset's `rule_1_violation` field (and the equivalent fields for rules 2 and 3) marks that *a specific worker* in the image lacks the required PPE or protection. `<OPEN_VOCABULARY_DETECTION>`, as used in v1, answers a *scene-level existence* question instead — "does a hard hat appear anywhere in this image?" Most ConstructionSite images contain multiple workers. If even one worker in the frame is wearing a hard hat, the existence query returns a box, and the v1 proxy answers "compliant," even when a different worker in the very same frame is the one the dataset actually flagged as the violation.

Concrete traced case. Image 79 (ground truth `fall_hazard`, rule 3, missing edge protection) makes the mechanism concrete. The "guardrail" query matched the wooden formwork bordering the excavation pit — a structure that is guardrail-shaped but is not a guardrail — and so the proxy answered "compliant," even though the dataset's actual violation is a missing barrier elsewhere in the same scene. The detection itself was not wrong in any technical sense: a guardrail-shaped object genuinely is present in the image. The query was simply answering a different, easier question than the one the ground-truth label was actually recording.

Why rule 4 was unaffected. The rule 4 proximity proxy does not share this failure mode, because it is relational rather than existential: it asks about the spatial relationship between two specific grounded objects (worker and excavator), not whether a safety object exists anywhere in the scene. This is consistent with rule 4 already showing real, balanced discriminative signal (62% sensitivity, 42% specificity) before any redesign was attempted.

Resolution, stated as a decision. Rather than silently patch around individual failure cases such as image 79, the rules 1–3 proxy was redesigned, as an explicit and documented decision rather than an incidental bug fix, to a **person-relative** check. The redesigned `_answer_presence_rule` detects all `"worker"` boxes and all relevant safety-object boxes independently, then flags **violation** if any one detected worker has no nearby or overlapping safety-object box, via a new `regions.all_boxes_covered` helper. This mirrors the dataset's own semantics directly: one non-compliant worker is enough for the dataset to mark the whole image violated, so the proxy now requires *every* detected worker to be covered before it will answer "compliant," rather than requiring only that the safety object exist somewhere in the frame. Worn-PPE rules (rule 1, hard hat; rule 2, harness) use a tight proximity threshold, `PRESENCE_PROXIMITY_FRACTION = 0.08` (8% of the image's maximum dimension), reflecting that the safety object must be on the worker's body to count as worn. Rule 3's guardrail is structural and can protect a worker from further away, so it uses a looser threshold, 0.20 (20%). Images with zero detected workers — 12 of the 163 samples — fall back to the original v1 scene-level existence check, since the person-relative logic has no worker box to anchor to in those cases.

3.7 Standard vs. Non-Standard Choices

Two distinct design decisions are at work in this chapter, and they sit at opposite ends of the standard/non-standard spectrum. **Standard**: treating a yes/no safety-compliance question as a closed-vocabulary grounding or detection task is a documented workaround pattern for vision-language models that, like Florence-2, lack a native free-form VQA head — reframing the question in terms of the model's actual task vocabulary, rather than forcing an unsupported task onto it, follows established practice for this class of model.

Non-standard, and the chapter's central methodological contribution: the person-relative redesign of `_answer_presence_rule` itself. This is not a textbook XAI or VQA technique drawn from the literature; it is a domain-specific adaptation that mirrors the ConstructionSite dataset's own per-worker labeling semantics — one non-compliant worker marks the whole image violated — and encodes that semantics directly into the proxy's logic via independent worker and object detection plus a per-worker coverage check. It was arrived at only after the v1 proxy's near-total sensitivity failure was diagnosed and traced to a specific, concrete case (image 79), not designed in advance. We report it here as a decision, with its quantified before/after effect, rather than folding it silently into the baseline as if it had always been the design.

3.8 Limitations

The person-relative redesign is bounded by a residual limitation in Florence-2's detection behavior that the redesign itself cannot work around. `<OPEN_VOCABULARY_DETECTION>` queried with the singular phrase `"worker"` returns only *one* box, even in scenes that visibly contain several workers. This was confirmed visually on two of the sample's own images: image 7 (five workers visible, ground truth `ppe_violation`, shown in Figure 3.2) and image 79 (multiple people visible, ground truth `fall_hazard`, the same image traced in Section 3.6). When the single worker box that Florence-2 happens to return is the compliant one, the per-worker coverage check has nothing to flag, even though the dataset's ground-truth violation concerns a different, undetected worker in the same frame. This caps how far the person-relative redesign can push sensitivity using the current detection strategy: the v2 proxy can only be as discriminating as the worker boxes it is given, and a single-box-per-scene detection will always miss multi-worker violations where the detected worker is not the violating one.

A different detection strategy — for example, `<DENSE_REGION_CAPTION>`, which is designed for exhaustive instance enumeration rather than single best-match grounding — could plausibly close this residual gap by returning all worker instances rather than one. This was deliberately *not* pursued in this pass: the 9x sensitivity gain already achieved by the person-relative redesign is a substantial, honestly-earned improvement in its own right, and chasing the residual gap immediately risked conflating two separate questions (is the person-relative logic correct, and is the underlying detection exhaustive) within a single change. The gap is documented here instead as an explicit, bounded limitation for future work.

3.9 Implications for the Paper

The headline number for this chapter, and the one a paper reviewer will most want explained clearly, is the 3.0%-to-27.0% (9x) sensitivity improvement in the rules 1–3 proxy, achieved by recognizing that the dataset's ground truth is a per-worker judgment and redesigning the proxy to match that judgment rather than continuing to ask a scene-level existence question. This redesign is not incidental to the pilot's later chapters: it is a precondition for them. Descriptive Accuracy, Robustness, and Bounded Completeness, evaluated in later chapters, all assume that the Day 3 baseline answer is

a meaningful judgment worth testing an explanation method against. Under the *old* v1 proxy, for rules 1–3, that assumption was nearly false — a near-constant "compliant" classifier gives a masking- or perturbation-based explanation method nothing to flip, because there is almost no correct "violation" judgment in the first place for an explanation to be faithful to. This is precisely why the redesign was carried out on Day 3, before any of those later evaluation days ran, rather than discovered and patched retroactively after building explanation methods on top of a proxy that could not discriminate.

For the eventual paper, we recommend a dedicated "Threats to Validity" paragraph addressing the residual detection gap documented in Section 3.8: that Florence-2's open-vocabulary detection for the singular phrase `"worker"` returns at most one instance per scene, and that this bounds the person-relative proxy's achievable sensitivity independently of how well its per-worker coverage logic is designed. Reporting this limitation alongside the 9x improvement allows the improvement to be read accurately — as a genuine, substantial gain that is nonetheless capped by an instance-enumeration constraint in the underlying vision-language model, rather than as a fully solved problem. With a baseline proxy now genuinely discriminating between compliant and violating images, the pilot's next step is to extract and evaluate the explanations that accompany these judgments.

Chapter 4

Standardizing Explanation Regions for Masking-Based Tests (Day 4)

4.1 Motivation

Day 3 left every sample with a baseline judgment and, for rules 1–3, a person-relative detection process that grounds independent worker and safety-object boxes rather than asking a single scene-level existence question. Those boxes were produced to decide compliant/violation; they were not yet organized into anything a later explanation-testing method could consume. Descriptive Accuracy, the masking-based evaluation planned for Day 5, needs a single, well-defined region to mask per sample — a candidate answer to "which part of the image does the model's judgment depend on?" — and a procedure for actually producing the masked image. Day 4's task was to build that standardization layer: take Day 3's already-computed boxes, rank them into an ordered list of candidate explanation regions, and provide a masking primitive that blacks out (or blurs) the top-ranked region for re-inference. Day 3 and Day 4's work were developed together, per commit `6de76fb`, reflecting that the redesigned proxy and the regions layer that consumes its boxes were designed as a single piece of work rather than as two independent days.

4.2 Implementation

`regions.py` introduces a `Region` dataclass — a `box`, a `source` field (`"model"` or `"grid"`), and a `label` — as the standard unit the rest of the pilot's masking-based tests operate on. `standardize_regions(boxes, labels, image_size, grid=(4, 4))` is the layer's central function: it ranks model-returned boxes by **area, descending**, and returns them as a list of `Region` objects with `source="model"`. Florence-2's grounding output carries no attention or saliency map, so there is no real per-region importance score available to rank by; area is used as an explicit, acknowledged stand-in, on the assumption that a larger detection is more likely to be the salient one than a sliver. Only when the model returned *zero* boxes at all does `standardize_regions` fall back to an unranked 4×4 grid, via `grid_fallback_regions`, so that every sample still has some candidate region to mask even when grounding failed outright.

Three smaller helpers round out the module: `iou()` computes intersection-over-union between two boxes; `mask_region()` fills a given box with either a solid black patch or a Gaussian-blurred copy of the same region (`mode="black"` or `mode="blur"`); and `boxes_overlap_or_close()` tests whether two boxes overlap or have edges within a distance threshold. The last of these underlies `all_boxes_covered()`, which is, in fact, the function that Chapter 3's redesigned `_answer_presence_rule` actually calls to check that every detected worker is covered by a nearby safety-object box. That helper's home in `regions.py` rather than in Day 3's own module is the clearest evidence that Day 3's proxy redesign and Day 4's regions layer were not sequential, independent pieces of work but a single designed unit, split across two chapters here only for narrative pacing.

`scripts/04_extract_regions.py` applies `standardize_regions` to all 163 samples, reusing the `worker_boxes` and `object_boxes` already written to `results/baseline_predictions.csv` by Day 3 — no new GPU inference was needed for this step. For each sample it records the top-ranked region's source and label, and for the first ten samples it additionally saves a before/after masked-image pair, applying `mask_region` in `"black"` mode to the top region, so the masking step itself can be sanity-checked visually before Day 5 relies on it.

4.3 Architecture: A New Masking Layer

Figure 4.1 shows where this chapter's layer sits relative to Day 3's baseline. Day 3 produces grounded boxes and labels; Day 4 adds everything downstream of that point — ranking the boxes into an ordered region list, and masking the top-ranked region to produce an image suitable for the re-inference step that Descriptive Accuracy will perform on Day 5. Nothing in this diagram changes how Day 3's boxes are produced; it only standardizes what is done with them afterward.

Figure 4.1: Regions/masking layer added on top of Day 3's baseline grounding output.

4.4 Worked Example

Figure 4.2 revisits image 7's rule 1 (hard hat) sample, the same worked example used in Chapter 3, for narrative continuity. The left panel shows the top-ranked region — the largest of the model-returned boxes, outlined for inspection — overlaid on the original image; the right panel shows the result of `mask_region` applied to that same box in `"black"` mode. The masked region corresponds exactly to the ranked top box and nothing else in the frame, confirming that the masking primitive is doing precisely what it is meant to do before it is relied on for the Descriptive Accuracy measurements in Chapter 5.

(a) Before: top-ranked region outlined.

(b) After: top-ranked region masked.

Figure 4.2: Image 7, rule 1 (hard hat): the region ranked first by area, before and after `mask_region` blacks it out.

4.5 Results

Running `standardize_regions` over all 163 samples shows that a real model-returned box was available and ranked first for **159/163 samples (97.5%)**; the remaining **4/163 (2.5%)** returned zero boxes and correctly fell back to the unranked grid. The model-box rate this high confirms that Day 3's grounding calls are, for the overwhelming majority of samples, supplying the regions layer with something real to rank, and that the grid fallback is functioning as the rare exception it was designed to be rather than as a frequently-used default. Visual inspection of 2 of the 10 saved before/after pairs confirmed that `mask_region` blacks out exactly the ranked top box and nothing else in the frame, as shown in Figure 4.2.

4.6 Problems Encountered and Resolutions

The central design problem on Day 4 was not a bug but a missing capability in the underlying model: Florence-2's grounding output provides no real saliency signal, no per-pixel or per-region importance score of the kind that an attention map or a gradient-based attribution method would supply. Without such a signal, there was no principled way to rank multiple candidate boxes by "how much the model's judgment depends on this region." The resolution was an explicit design decision, made and documented here rather than left implicit, to rank by box area as the best available stand-in — on the working assumption that a larger detected region is more likely to be the one driving the judgment than a small one — while flagging plainly that this is a stand-in, not a claim of true importance. **This choice is shown later in the pilot (Chapters 6, 7, and 10) to systematically disadvantage small PPE objects**: a hard hat or harness is small relative to the worker or scene around it, so an area-based ranking will tend to rank larger, non-PPE boxes above the actual PPE object whenever both are present as candidates. That consequence is forward-referenced here, at the point the choice was made, so it reads in this report as a foreseen limitation rather than one discovered by accident downstream.

4.7 Standard vs. Non-Standard Choices

Standard: using bounding-box masking or occlusion as an explanation-testing primitive is a long-established pattern in vision explainability, the same basic idea behind occlusion sensitivity maps — block out a region, observe whether and how the model's output changes, and treat a change as evidence that the region mattered. Adopting that primitive here, rather than inventing a new testing mechanism, follows established practice.

Non-standard: ranking candidate regions purely by box area as a substitute for a true importance score is not a published method, and is not presented as one. It is an explicit, acknowledged simplification, adopted because Florence-2's grounding output leaves no real saliency signal to rank by, and reported here as a designed-and-flagged limitation rather than folded silently into the pipeline as if it were equivalent to a genuine importance ranking.

4.8 Implications for the Paper

The area-ranking heuristic should be flagged explicitly in the paper as a limitation and future-work item: it is the single highest-leverage fix identified across the entire pilot (see Chapter 10, Finding 1, and Chapter 14's task list, item 3). Its effect is not confined to this chapter. Because `standardize_regions`'s ranking feeds every later masking-based test built on top of it, the area-based choice made here is the mechanism behind the PPE-disadvantage findings reported in Chapters 6, 7,

and 10: small, genuinely safety-relevant objects are structurally under-ranked relative to larger, often less relevant boxes, independent of how well any later evaluation metric itself is designed. Recording that link here, at the point of the original design decision, lets the paper present the limitation as anticipated and explained rather than as a finding that surfaces without an account of its cause.

Chapter 5

Descriptive Accuracy: Does Masking the Top Region Change the Answer? (Day 5)

5.1 Motivation

Day 4 produced, for every sample, a ranked list of candidate explanation regions and a masking primitive capable of blacking out the top-ranked one. Neither is useful on its own: ranking a region first and being able to mask it says nothing yet about whether that region actually mattered to the model's judgment. Descriptive Accuracy closes that gap. It is one of the six metrics defined for evaluating black-box explainable-AI frameworks [3], and the idea behind it is simple and falsifiable: if the explanation method has correctly identified the region the model's decision depends on, removing that region should be capable of changing the decision; if masking the region never changes anything, the region was not load-bearing for the answer in the first place, and the explanation pointing to it was not describing anything the model actually used. Day 5's task was to run this test, for the first time in the pilot, against the redesigned proxy and the standardized regions that Days 3 and 4 built specifically to support it.

5.2 Implementation

`descriptive_accuracy.py`'s `evaluate()` function implements the test directly on top of the two modules already built: `inference.py`'s `answer_rule` and `regions.py`'s `mask_region`. For each sample, it masks the top-1 standardized region (black-fill) on the original image, reruns `answer_rule`, and compares the result to the unmasked baseline answer already on record from Day 3. It then proceeds further along the same ranked list: starting again from the original image, it masks the top-1 region *and* the top-2 region together, reruns `answer_rule` a second time, and again compares to baseline. No new pipeline stage was built for this chapter; Descriptive Accuracy is a composition of two modules that already existed, run twice per sample with progressively more of the image blacked out.

`scripts/05_descriptive_accuracy.py` drives this evaluation over all 163 pilot samples, reusing the `worker_boxes` and `object_boxes` already written to `results/baseline_predictions.csv` on Day 3 to reconstruct each sample's standardized regions exactly as Day 4 ranked them, then calling `evaluate()` once per sample. Results are written to `results/descriptive_accuracy.csv`, one row per sample recording the baseline answer, the masked answer under each masking level, and whether each masking level flipped the answer relative to baseline. For the first ten samples whose top-1 masking flips the answer, the script additionally renders an overlay image showing exactly which region was masked, saved to `results/figures/descriptive_accuracy/`, so that flip cases can be inspected visually rather than taken only on the strength of a boolean column in a CSV.

5.3 Results

Across all 163 samples, masking the top-1 region flips the answer in **36.2%** of cases. Masking the top-1 and top-2 regions together flips the answer in **35.0%** of cases — a *lower* rate than masking top-1 alone, an anomaly examined in its own section below.

Masking level	Descriptive accuracy (answer flip rate)
Top-1 region	**36.2%**
Top-1 + top-2 regions	35.0%

5.3.1 By primary class

Flip rate under top-1 masking varies substantially by ground-truth `primary_class`: `struck_by_risk` flips most often, at 61.5%, followed by `fall_hazard` at 46.0%; `ppe_violation` and `compliant` both flip at 28.0%.

Primary class	Top-1 flip rate
struck_by_risk	61.5%
fall_hazard	46.0%
ppe_violation	28.0%
compliant	28.0%

5.3.2 By assigned rule

Broken down by `assigned_rule_id` instead, top-1 flip rate is highest for rule 4 (44.0%) and rule 3 (42.9%), lower for rule 2 (38.5%), and lowest for rule 1 (27.0%).

Assigned rule	Top-1 flip rate
rule_4 (struck-by risk)	44.0%
rule_3 (guardrail)	42.9%
rule_2 (harness)	38.5%
rule_1 (hard hat)	27.0%

This per-rule ordering tracks Day 3's redesigned-proxy sensitivity ordering closely, and the correspondence is not coincidental. Day 3 found rule 4's proximity check and rule 3's looser-threshold guardrail check carrying the strongest real discriminative signal among the four rules, with rule 1's hard-hat check the weakest (16.0% sensitivity). The same ordering reappears here: a proxy that barely discriminates between compliant and violating inputs in the first place also has comparatively little for masking to disrupt, because so much of its output is already determined by something other than the specific region being tested. A proxy with genuine discriminative signal, by contrast, is more exposed to having that signal's load-bearing region removed, and so flips more often under masking. Descriptive accuracy, in other words, is not an independent measurement here so much as a second view of the same underlying proxy quality Day 3 already characterized.

5.4 Worked Examples

Two of the ten saved flip overlays illustrate the range of behavior masking can produce, from the mechanistically subtle to the intuitively clean.

Figure 5.1: Image 69, rule 1 (hard hat): the top-1 region is the worker box, not the hard-hat box. Masking it removes the worker from detection entirely, rerouting the answer through a fallback branch rather than removing direct evidence about the hard hat — the case traced in detail in Section 5.5.

Figure 5.2 is the intuitive case, and it is worth stating explicitly what makes it intuitive: the masked region is exactly the rule-defining object, and removing it removes the hazard signal in a way that matches what "explanation" should mean. Rule 4's proximity check depends on two grounded objects, a worker and an excavator; masking the excavator box removes one of the two quantities the proximity comparison needs, and the answer flips from `hazard` to `safe` because the check can no longer establish that the worker is close to anything dangerous. This is descriptive accuracy behaving exactly as the metric is designed to detect: the top-ranked region was, in fact, the region the judgment depended on, and masking it changed the judgment for a reason that tracks the underlying safety concept.

Figure 5.1 looks superficially similar — an answer flip following a masked top-1 region — but the masked region is the *worker* box, not the hard-hat box, and the mechanism behind its flip is entirely different from Figure 5.2's. That mechanism is the subject of the next section.

5.5 The Non-Monotonicity Anomaly

Naively, masking *more* of an image's evidence should never un-flip an answer that masking *less* had already flipped: if removing one region was enough to change the model's judgment, removing that

Figure 5.2: Image 120, rule 4 (struck-by risk): the top-1 region is the excavator box itself. Masking it removes the proximity check's other half, flipping `hazard` → `safe`.

region and another should still leave the judgment changed, since nothing has been added back. The headline numbers above show this expectation failing outright. Top-1+top-2 masking's flip rate, 35.0%, is *lower* than top-1 masking's flip rate, 36.2%, and the difference is not sampling noise: in 15 of 163 samples, masking the second-ranked region in addition to the first reverses a flip that masking the first region alone had already produced, restoring the original baseline answer.

5.5.1 The traced case: image 0000069, rule_1

Image `0000069` (rule 1, hard hat) was traced in full to establish the mechanism behind these 15 regressions concretely rather than dismissing them as noise.

Baseline. The sample's baseline detection returns two boxes: one worker box, with area 10,778 px^2, and one hard-hat box, with area 891 px^2. The worker box is more than twelve times larger than the hard-hat box. Day 4's `standardize_regions` ranks candidate regions by area, descending, with no other signal available to rank by — so the worker box, not the hard-hat box, is ranked top-1, and the hard-hat box is ranked top-2.

Masking top-1 (the worker). With the worker box blacked out, Florence-2's worker-detection call on the rerun returns no boxes at all: `worker_boxes` comes back empty. This triggers the `if not worker_boxes` branch in `_answer_presence_rule` (`inference.py`), the same fallback path Chapter 3 introduced for samples with zero detected workers: "no worker detected, so answer `compliant` if the safety object exists anywhere in the frame, `violation` otherwise." The hard-hat box is still visible — only the worker region was masked — so the fallback finds an object and answers **compliant**. Baseline's

answer for this sample was `violation`, so this is recorded as a flip.

Also masking top-2 (the hard hat). Starting over from the original image and masking both the worker box and the hard-hat box together, neither region is visible to the rerun. The same fallback branch fires again — there is still no detected worker — but this time the scene-level existence check for the safety object also fails, because the hard hat is now hidden too. The fallback therefore answers **violation**: the original baseline answer. Masking strictly more of the image has reverted the answer to where it started.

The mechanism, generalized. This is a real interaction between the masking test and the person-relative redesign's own fallback logic from Chapter 3, not a bug in either component considered alone. Masking the *worker* region does not simply remove one piece of evidence among several that the per-worker coverage check weighs; it removes the precondition, `worker_boxes` being non-empty, that the per-worker check needs even to run. Once that precondition fails, control silently reroutes to the cruder scene-level branch, whose answer then depends only on whether the *other* region (the safety object) happens to still be visible. That dependency is binary and is not monotonic in how much of the image has been masked: visible-object → `compliant`, masked-object → `violation`, regardless of whether one region or two has been blacked out. Masking top-1 alone leaves the object visible and so reads as `compliant`; masking top-1 and top-2 together hides the object too and so reads as `violation` again — and because the rule's binary answer space has only these two values, hiding the object after the worker is already gone does not produce some third intermediate answer, it produces the literal original baseline value. All fifteen regressions follow this exact full-revert pattern: `masked_answer_top2` equal to `baseline_answer`, not merely unequal to `masked_answer_top1`[1]. Confirmed visually for `0000069` in Figure 5.1, the masked region drawn there is unambiguously the worker box, not the hard-hat box, exactly as the traced mechanism requires.

5.5.2 Why this is not a defect to patch

It would be tempting to treat this as a bug to fix — a "broken" fallback branch that should be removed or special-cased. That would be the wrong response. The fallback branch exists because Day 3 needed some defined answer for the 12 of 163 samples with zero detected workers at baseline; it is not incidental code, it is a deliberate, documented design decision serving a real and separate purpose. The anomaly here is not that the fallback branch is wrong, but that the masking test, by construction, can manufacture exactly the condition the fallback branch was built to handle — zero detected workers — as a side effect of removing a region for an unrelated reason, namely testing whether that region was load-bearing for the answer. The two pieces of logic are each correct on their own terms; the anomaly is what happens when one test's side effect collides with the other's precondition.

5.6 Standard vs. Non-Standard Choices

Standard: deletion- or masking-based descriptive accuracy is a well-established explanation-fidelity pattern in the explainability literature, and is adopted here directly as one of the six metrics defined for evaluating black-box XAI frameworks [3] — that work is the direct methodological source for the metric as implemented in this chapter, rather than a method invented for this pilot.

Non-standard: discovering and mechanistically tracing the non-monotonic "masking more un-flips the answer" anomaly to a fallback-branch interaction, rather than treating the 15 affected samples

[1] Recomputed directly from `results/descriptive_accuracy.csv`: of the 15 samples with `answer_changed_top1=True` and `answer_changed_top2=False`, `masked_answer_top2` equals `baseline_answer` for all 15, not 14 as an earlier informal note on this finding stated.

as unexplained noise or excluding them from the headline numbers, is this chapter's contribution. The anomaly was run down to a specific, concrete case (`0000069`) and a specific line of code (`if not worker_boxes` in `_answer_presence_rule`), and reported as a real interaction between two correct pieces of logic rather than papered over as measurement error.

5.7 Implications for the Paper

The headline 36.2%/35.0% pair should be reported together, not as two independent numbers but as a single finding: that the second number is *lower* than the first is the more important fact about this chapter's results than either number's absolute size. We recommend the paper state the non-monotonicity finding as a generalizable diagnostic, applicable beyond this one pilot: **watch for non-monotonic masking results as a sign of hidden fallback logic.** Any masking- or deletion-based explanation-fidelity test, run against any pipeline with conditional branches, is capable of producing this same pattern whenever the masking procedure can remove the precondition for one branch and silently reroute execution to another — the non-monotonicity is not specific to Florence-2, to construction safety, or to this particular fallback, but to the general shape of testing-by-deletion against branching logic.

This implication is flagged forward explicitly because it recurs later in the pilot under a different name. Robustness (Chapter 9) and Bounded Completeness (Chapter 10) both rerun `answer_rule` on modified images and compare the result to baseline — the same shape of test as this chapter, applied to perturbations and to completeness checks rather than to masking. The same fallback-rerouting risk applies there: a perturbation or mask that happens to remove the only detected worker, rather than the safety object the rule is nominally about, can flip the answer for reasons disconnected from the safety concept the rule is meant to test. We recommend that Chapters 9 and 10 check explicitly for this pattern in their own results — inspecting whether flips correlate with a worker box dropping out of detection — rather than reporting only an aggregate flip or completeness rate and treating any non-monotonic or counter-intuitive cases within it as unexplained.

Chapter 6

Visual Sparsity via Decoder-Encoder Cross-Attention: Why Attention Rollout Does Not Apply (Day 6)

6.1 Motivation

Days 3–5 characterized the model's grounded boxes and tested whether masking the top-ranked one changes the answer, but neither day asked the more direct visual-attribution question: when Florence-2 grounds a phrase, where in the image does it actually attend while doing so? Visual sparsity is one of the six metrics defined for evaluating black-box explainable-AI frameworks [3], and it answers exactly that question — given a per-pixel or per-patch attribution signal, how concentrated is the attention mass that signal assigns across the image? A sparse, concentrated heatmap is the visual signature of a focused explanation; a heatmap spread thinly and evenly across the frame carries comparatively little information about where the judgment actually came from. Day 6's task was to produce that attribution signal for Florence-2 and score its concentration across the pilot's samples. The plan, written before this day's implementation began, specified a particular method for producing the signal: attention rollout. That plan did not survive contact with Florence-2's actual architecture, and the deviation that resulted — discovered, verified, and resolved before any code was written against the wrong assumption — is this chapter's centerpiece.

6.2 Method Deviation from the Plan

The plan called for `attention_rollout()` first, the textbook ViT-style explainability method that recursively composes self-attention matrices across the layers of a single-modality self-attention-only encoder to estimate how much each input token ultimately propagates into each output token [1]. Before implementing it, Florence-2's actual cached modeling code (`modeling_florence2.py`) was checked directly, rather than assuming a textbook ViT-style rollout would apply unmodified — and it does not apply cleanly.

The verified architecture fact. Florence-2's encoder does not run a self-attention stack over image patches alone. It runs **one shared self-attention stack over a fused image+text sequence**: `[1 global "spatial_avg_pool" token] + [576 image-patch tokens, 24×24] + [text prompt tokens]`. This was confirmed directly against the model's own code, not inferred from documentation or assumed by analogy to a generic vision transformer. Two specific checks established it:

- `_encode_image` returns `torch.cat([spatial_avg_pool_x, temporal_avg_pool_x])`, which is $1+576 = 577$ tokens, confirmed by printing `image_features.shape` directly rather than reading the count off a docstring.
- $576 = 24 \times 24$ follows from the DaViT vision tower's four convolutional stages (strides 4, 2, 2, 2)

applied to the processor's fixed 768×768 input image: $768 \to 192 \to 96 \to 48 \to 24$, computed by hand from the stride arithmetic rather than taken on faith.

Why this breaks rollout. Classic attention rollout [1] is defined for a self-attention-only stack processing *one* modality: the method's recursive matrix-composition step assumes that what is being mixed across layers is attention among tokens of a single, homogeneous kind, so that the result can be read as "how much does input patch i ultimately influence output patch j." Florence-2's encoder violates that assumption at its foundation: every self-attention layer in the fused stack mixes image-patch tokens with text prompt tokens at the same time, in the same attention matrix. Rolling out Florence-2's encoder as if it were a single-modality stack would recursively mix in attention to text tokens at every layer in a way the method was never designed for, and that mixing is not separable back out after the fact — there is no post-hoc operation that can take the rolled-out, jointly-mixed result and cleanly attribute the image-only portion of it. The architecture was checked first, specifically because the plan's chosen method carries this hidden precondition, and the check showed the precondition fails.

What was used instead. Decoder→encoder cross-attention was substituted for rollout. For each token the decoder generates, it attends back to the full 577-token encoder sequence; `attribution.py`'s `cross_attention_heatmap()` averages this cross-attention over decoder layers, heads, and generated tokens, then drops the global token (index 0) before reshaping the remaining 576 values into the 24×24 patch grid. Unlike rollout, cross-attention requires no recursive composition and no single-modality assumption: it is a per-step, already-computed quantity — "where did the decoder look, this step, to produce this output token" — read directly off the model rather than approximated by composing layers. This substitution is well-established for encoder-decoder transformers generally (cross-attention visualization has long been used for machine-translation attribution) and sidesteps the fused-modality problem entirely, because cross-attention is always computed against the same fixed encoder sequence regardless of how that sequence's internal self-attention mixed modalities together.

Other attribution families were also considered and set aside for this architecture, a point returned to in Section 6.8: gradient- and perturbation-based attribution methods of the kind implemented in Captum [5], layer-wise relevance propagation [4], and DeepLIFT [7] all could, in principle, be adapted to Florence-2, but each carries its own architectural assumptions that would need the same kind of verification rollout was given here, and none was implemented or verified within this pilot's scope. Notably, these are exactly the white-box methods Abdallah's own prior work has compared directly against each other — layer-wise relevance propagation, integrated gradients, and DeepLIFT, evaluated on tabular network-intrusion DNNs [2] — so adapting any of them to a fused-modality VLM encoder is a genuine extension of that line of work, not a straightforward port.

6.3 Architecture: An Attribution Layer

Figure 6.1 lays out the attribution pipeline that resulted from Section 6.2's deviation. Generation for attribution purposes always runs in greedy mode (`num_beams=1`), a difference from Days 3–5 taken up in Section 6.4. The decoder's cross-attention to the full 577-token encoder sequence is captured at every generation step, averaged over layers, heads, and generated steps into a single 577-length vector, then the global token is dropped and the remaining 576 values are reshaped into the 24×24 patch grid and normalized to $[0, 1]$ before the sparsity metrics are computed on it.

Nothing about this pipeline reaches back into Florence-2's encoder self-attention — the fused-modality problem identified in Section 6.2 is avoided entirely by operating only on the cross-attention edge between decoder and encoder, a quantity the fused encoder's internal mixing does not touch.

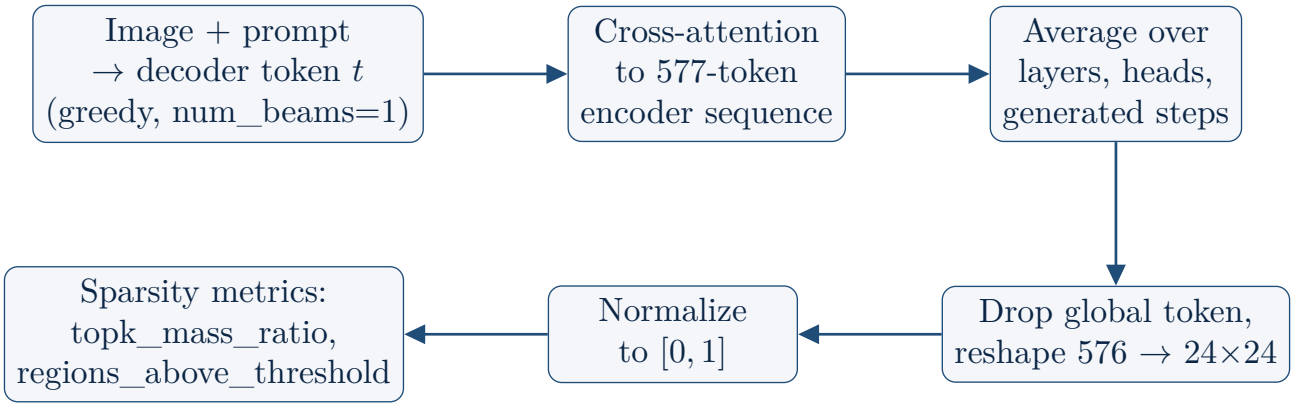


Figure 6.1: Attribution layer: decoder→encoder cross-attention, substituted for attention rollout.

6.4 Implementation Details

`attribution.py`'s `cross_attention_heatmap()` implements Figure 6.1's pipeline directly. Two implementation details were significant enough to flag explicitly rather than leave implicit in the code.

The off-by-one trap. 577 is not a perfect square. Computing $\sqrt{\texttt{num\_image\_tokens}}$ directly — the natural first move for reshaping a sequence of attention weights into a square grid — would have silently misaligned every heatmap by one cell, because the result, $\sqrt{577} \approx 24.02$, is not an integer and gives no warning that something is wrong before producing a plausible-looking but wrong grid. The actual fix relies on the architecture fact verified in Section 6.2: index 0 of the 577-token sequence is the global `spatial_avg_pool` token, not a patch, so it must be dropped *before* reshaping, leaving exactly 576 values for a clean 24×24 grid. This is flagged here as a trap rather than a routine implementation step because it would have failed silently — the reshape would not raise an error on a near-square count, it would simply produce a heatmap shifted by one cell relative to the true patch layout, with no symptom visible short of comparing the result against an independently verified ground truth.

Decoding-mode difference from Chapters 3–5. Cross-attention extraction always uses `num_beams=1` (greedy decoding), not the `num_beams=3` beam search used for the logged answers in Chapters 3–5. This is a deliberate substitution, not an oversight: Hugging Face's beam search reorders the batch dimension at every generation step to keep the surviving beams contiguous, and `generate()`'s returned attentions are not automatically un-reordered to compensate, so beam-search cross-attentions do not line up sample-by-sample with the tokens they nominally belong to without extra bookkeeping this pilot's scope did not require. Greedy decoding sidesteps the reordering problem entirely, at the cost of attributing a freshly greedy-decoded output rather than the original beam-search answer. The substitution was spot-checked rather than assumed safe: on image `0000007`, the greedy-decoded boxes matched Chapter 4's beam-search boxes closely in position. The substitution changes *how* the output was decoded, not *what* gets attributed — the same regions are grounded either way, which is what makes the cross-attention heatmap a meaningful attribution signal for the beam-search answers reported elsewhere in this pilot, even though it is computed against a greedy rerun.

6.5 Results

`scripts/06_visual_sparsity.py` ran the attribution pipeline over all 163 pilot samples. 4 of the 163 samples are Day 4's grid-fallback cases — no model-returned box at all, and therefore no grounded

phrase to attribute attention to — and were excluded from scoring, the same exclusion Day 4 itself flagged. Sparsity was scored on the remaining **159 samples**.

Metric	Mean (159 scored samples)
Top-5-of-576-cells mass ratio	**0.108**
Top-20-of-576-cells mass ratio	0.247
Cells above 0.5 threshold (of 576)	7.0

A top-5 mass ratio of 0.108 means the five highest-attention cells, out of 576 total, together carry only 10.8% of the total attention mass on average — a long way from the concentration a genuinely sparse, focused explanation would show, and a useful baseline against which the by-rule and by-phrase breakdowns below can be read.

6.5.1 By assigned rule

Assigned rule	Top-5 mass ratio
rule_1 (hard hat)	0.125
rule_2 (harness)	0.113
rule_3 (guardrail)	0.098
rule_4 (struck-by risk)	0.079

6.5.2 By attributed phrase

Attributed phrase	Top-5 mass ratio
hard hat	**0.195**
safety harness	0.118
worker	0.115
guardrail	0.099
excavator	**0.079**

Hard hat is the most concentrated phrase in the breakdown, at 0.195; excavator is the least concentrated, at 0.079 — the two phrases anchor opposite ends of the by-phrase ordering, and the gap between them is the subject of this chapter's central finding.

6.5.3 Finding: concentration is driven mainly by target physical size, not explanation quality

Hard hat's top-5 mass ratio, 0.195, is roughly **2.5**× excavator's, 0.079. The natural first reading of that gap is that the model's explanations for hard-hat judgments are substantially better-focused than its explanations for excavator-related judgments. The data does not support that reading once the grounded objects' own physical sizes are brought in. Mean detected box area per phrase, taken from `baseline_predictions.csv`, tracks the concentration ordering closely:

Phrase	Mean box area (px^2)	Top-5 mass ratio
excavator	287,913	0.079
guardrail	310,291	0.099
safety harness	178,491	0.118
worker	79,005	0.115
hard hat	94,330	0.195

The two largest objects, guardrail and excavator, are also the two least concentrated; the smallest object, hard hat, is the most concentrated. The correlation between log(box area) and top-5 mass ratio, computed across all scored samples, is -0.70 — a strong, negative relationship between how physically large the grounded object is and how concentrated the attention attributed to it appears.

This is a real confound, not a bug, and not a flaw in the attribution pipeline built in this chapter. A small object's 576-cell patch grid mechanically has fewer cells it *could* cover in the first place: a hard hat that occupies a handful of patches will, almost by construction, show its attention mass concentrated into a handful of cells, regardless of whether the underlying reasoning that produced that attention is any good. A guardrail spanning much of the frame has many more cells available to it geometrically, so even attention that is, in some qualitative sense, just as well-targeted will register as more spread out by a metric defined purely over cell counts. Visual sparsity, as measured here, partly reflects target footprint size rather than purely reflecting explanation focus, and that confound has to be read into any later comparison of sparsity across rules that attribute to objects of different physical sizes.

6.6 Worked Examples

Figure 6.2 shows the two saved overlays that anchor the by-phrase finding visually, one from each end of the concentration ordering.

(a) Image 23, rule 1 (hard hat): a small, sharp hotspot on a distant worker's hard hat.

(b) Image 79, rule 3 (guardrail): attention spread broadly across most of the frame.

Figure 6.2: The clearest focused and scattered examples in the sample, consistent with the hard-hat/excavator concentration gap and its physical-size explanation.

Image 23's heatmap is the clearest "focused" example in the sample: a small, sharp hotspot lands exactly on a distant worker's hard hat, against an otherwise empty dirt lot with nothing nearby to compete for attention mass. Image 79's heatmap is the clearest "scattered" example: attention spreads broadly across most of the frame, consistent with the guardrail's large, elongated structural footprint rather than with any failure of the model to localize it. Read side by side, the two panels make the size confound visually concrete — the hard hat is small and the heatmap is concentrated; the guardrail is large and the heatmap is not, and nothing about either heatmap looks like a worse or better explanation in any qualitative sense, only a smaller or larger target.

6.7 Standard vs. Non-Standard Choices

Standard: using decoder→encoder cross-attention as an attribution signal for an encoder-decoder transformer is itself well-established practice, the same basic technique behind attention visualization for machine translation — read the existing cross-attention weights directly off the model's forward pass, average appropriately, and treat them as evidence of where the model was looking when it produced a given output. Adopting that established technique here, rather than inventing a new mechanism for reading attention out of Florence-2, follows precedent.

Non-standard: the explicit, verified decision *not* to use attention rollout — the textbook ViT XAI method the plan originally specified — is the non-standard, and reportable, choice in this chapter. Most pipelines that adopt a planned method do so without re-checking whether the target model's actual architecture satisfies that method's assumptions; this pilot checked Florence-2's cached modeling code first, found that its fused image+text encoder breaks attention rollout's single-modality precondition, and substituted cross-attention instead, with the substitution's correctness established by direct inspection of the model's code rather than by analogy or convention. That the substitution was discovered, documented, and resolved before any rollout code was written against the wrong assumption is the genuine methodological deviation this chapter records.

6.8 Implications for the Paper

The architecture verification in Section 6.2 should be written up as a candidate dedicated subsection in the paper on **adapting attribution methods to fused-modality encoders**, not folded silently into an implementation appendix. The general lesson generalizes well beyond Florence-2: any vision-language model whose encoder processes image and text tokens through a single shared self-attention stack, rather than through separate modality-specific towers, will break attention rollout's single-modality assumption in the same way, for the same reason, and the fix — falling back to a decoder→encoder cross-attention signal that does not require composing self-attention layers at all — is also general, wherever the architecture provides a decoder that attends back to the encoder's output.

This generalization extends past rollout to the other attribution families considered and set aside in Section 6.2. Captum's perturbation- and gradient-based methods [5], layer-wise relevance propagation [4], and DeepLIFT [7] are, as published, also defined against architectural assumptions — clean gradient flow through a single tower, or a relevance-conservation rule defined layer by layer — that a fused-modality encoder like Florence-2's does not straightforwardly satisfy either, and none of the three was implemented or verified against this architecture within the pilot's scope. We recommend the paper state plainly that the cross-attention substitution made here is a documented, verified-correct *choice among constraints*, not evidence that rollout-style or gradient-based attribution is impossible for fused encoders in general — only that none of the textbook implementations applies without the same kind of architectural verification this chapter performed, and that verification, not assumption, is what

should precede any attribution method's adoption against a new architecture. This is also the natural place to surface Chapter 14's open question 1 to Professor Abdallah, on which attribution method, if any, the paper should treat as acceptable evidence for a fused-modality vision-language model when none of the standard methods applies in its textbook form.

Separately, the size confound identified in this chapter's central finding should be carried forward as a constraint on any cross-rule sparsity comparison later in the pilot: **comparing sparsity across rules without controlling for the grounded object's physical size will make large-object rules (rule 4's excavator) look artificially worse than small-object rules (rule 1's hard hat) by this metric alone**, independent of any real difference in explanation quality. Finally, the design decision to report visual and text attribution streams **separately, never merged into one score**, is carried forward from this chapter rather than introduced fresh later — the same separation is mirrored by Chapter 9's answer-level versus explanation-level robustness split, and recording the precedent here lets that later split read as a continuation of an established practice rather than as a new and unmotivated one.

Chapter 7

Stability Under Stochastic Decoding (Day 7)

7.1 Motivation

Stability is one of the six metrics defined for evaluating black-box explainable-AI frameworks [3]: rerun the same input through the model multiple times and ask whether the answer, and the region the answer is grounded in, stays put. Days 3–6 all ran Florence-2 exactly once per sample. Stability is the first day to ask what happens when the same sample is run more than once and the two runs are compared against each other — and that question carries a trap the plan flagged explicitly before any code was written against it.

The vacuous-result warning. Florence-2's default decoding configuration, used throughout Chapters 3–6, is beam search with `num_beams=3` and no sampling: deterministic decoding, in the sense that the same image and prompt, run through the same model weights, produce bit-identical output every time. Rerunning a deterministic pipeline three times and comparing the three outputs would report "100% stable" by construction, for every sample, regardless of whether Florence-2's underlying judgment is actually robust to anything. That result would be vacuous — true of any deterministic function applied to a fixed input, and informative about nothing. Taking this warning seriously up front, before treating any stability number as meaningful, is what shapes this chapter's implementation choice and its first reported check.

7.2 Implementation

`metrics/stability.py`'s `run_n_times()` avoids the vacuous case by switching decoding mode for the rerun comparison specifically: each sample is rerun 3× with `do_sample=True`, `num_beams=1`, `temperature=0.7` — genuine stochastic sampling, not the deterministic beam search used for the answers reported elsewhere in this pilot. The three sampled reruns are then compared pairwise to compute `answer_agreement_rate` (do the reruns agree with each other on the final yes/no safety-rule answer) and two region-overlap scores, both intersection-over-union (IoU) measures computed across the three reruns' top-ranked-region boxes. The deterministic beam-search answer that Chapters 3–5 already logged in `baseline_predictions.csv` is brought back in only as a separate comparison point — `deterministic_matches_majority` — never folded into the stability score itself, preserving the distinction between "how stable is the model under genuine resampling" and "does the model's one official answer happen to agree with the resampled majority."

7.2.1 Confirming the result is not vacuous

Before trusting any aggregate stability number, the sampled-rerun setup was checked directly against the failure mode it was designed to avoid: if every sample showed perfect agreement across all three

sampled reruns, that would be indistinguishable from the vacuous deterministic case, and would mean `temperature=0.7` sampling was not actually exercising any real variability in Florence-2's output. It is not vacuous. Mean `answer_agreement_rate` across all 163 samples is **0.775**, not 1.0 — roughly a quarter of rerun pairs genuinely disagree on the final answer. That gap from 1.0 is the evidence that stochastic decoding is doing real work here, and it is what licenses treating the rest of this chapter's numbers as a measurement of something real rather than an artifact of comparing a deterministic function against itself.

7.3 Results

`scripts/07_stability_test.py` ran the rerun pipeline over all 163 pilot samples: three sampled reruns each, plus the separate comparison against the existing deterministic answer.

Metric	Mean (163 samples)
`answer_agreement_rate` (pairwise answer match across 3 reruns)	0.775
`deterministic_matches_majority` (beam search vs. sampled majority)	83.4%
`top_region_overlap_score` (IoU of area-ranked top-1 region)	**0.667**
`object_region_overlap_score` (IoU of the rule's safety-object box)	**0.602**

`deterministic_matches_majority` at 83.4% says that Florence-2's one official, deterministic answer agrees with the majority answer across the three stochastic reruns for most, but not all, samples — a separate and reassuring signal that the deterministic pipeline used everywhere else in this pilot is not systematically an outlier relative to what the model would say under resampling, even though stability itself, 0.775, shows real rerun-to-rerun disagreement on roughly a quarter of samples.

7.3.1 By assigned rule

Rule	answer_agreement_rate	object_region_overlap_score
rule_1 (hard hat)	0.735	0.499
rule_2 (harness)	0.744	0.533
rule_3 (guardrail)	0.782	0.615
rule_4 (excavator proximity)	0.893	**0.922**

The by-rule breakdown already shows a pattern worth flagging on first read: rule_4's excavator answers and object boxes are both noticeably more stable than rules 1–3's, and the gap is far larger for `object_region_overlap_score` (0.499 at the low end to 0.922 at the high end) than for `answer_agreement_rate` (0.735 to 0.893). The remainder of this chapter is about that wider gap, and about the fact that the headline `top_region_overlap_score` — 0.667, reported as a single aggregate above — does not show it at all.

7.4 Finding: A Hidden Instability

`top_region_overlap_score`'s 0.667 mean looks reasonably stable. It is hiding how unstable the safety-relevant box actually is. The metric is computed on Day 4's area-ranked top-1 region —

the same `standardize_regions` ranking introduced in Chapter 4, which orders candidate boxes by pixel area with no access to a true importance signal. For rules 1–3, the area-ranked top-1 region is almost always the **worker** box, simply because a worker's bounding box is much larger in pixels than the hard hat, harness, or guardrail the rule is actually asking about. `top_region_overlap_score` at 0.667, in other words, is substantially a measurement of how stable Florence-2's worker-detection box is across stochastic reruns — informative in its own right, but not a direct answer to the question this chapter actually needs answered: is the explanation grounded in the rule's safety object stable?

To answer that question directly, `object_region_overlap_score` was added as a *separate* metric, computed specifically on `object_boxes[0]` — the rule's actual safety-object box, not whichever box happens to rank first by area. The gap this exposes is large. Rule_1's hard-hat box overlaps only **0.499** across reruns, against rule_4's excavator box at **0.922** — nearly **2**× more stable. Averaged across all 163 samples, `object_region_overlap_score` comes in at **0.602**, noticeably below `top_region_overlap_score`'s 0.667 — a smaller gap in the aggregate than the by-rule breakdown shows, because rule_4's excavator pulls the object-score average up even as rules 1–3 pull it down. The aggregate alone understates how unevenly that instability is distributed; the by-rule table is where the real picture is.

7.4.1 Ruling out box reordering

An easy alternative explanation needed to be checked and ruled out before attributing this gap to anything more interesting: could `object_region_overlap_score`'s instability simply be multiple detected boxes getting *reordered* across stochastic runs, so that `object_boxes[0]` on one rerun picks out a genuinely *different* detected box than `object_boxes[0]` on another rerun — a list-indexing artifact, not evidence that any one box is actually *moving*? This was checked directly rather than assumed away: only **4–8%** of samples have more than one object box detected at all (rule_1 6.3%, rule_2 3.8%, rule_3 8.2%, rule_4 4.0%). With reordering only possible in a single-digit-percentage slice of samples per rule, it cannot explain a systematic, roughly 2× gap in mean IoU that holds across the full 24–63 samples assigned to each rule. The instability is real box movement, not list-index noise.

7.4.2 The real driver: object size and shape

With reordering ruled out, the explanation that survives is a combination of **object size and shape** — the same kind of confound Chapter 6 identified in the visual-sparsity metric, here showing up in a different metric for a related reason. Median detected box area, taken from the sampled reruns, lines up closely with `object_region_overlap_score`:

Rule	Object	Median box area (px^2)	object_region_overlap_score
rule_1	hard hat	570	0.499
rule_2	harness	7,169	0.533
rule_3	guardrail	221,163	0.615
rule_4	excavator	215,069	**0.922**

The two smallest objects, hard hat and harness, are also the two least stable; the two largest, guardrail and excavator, are the two most stable. Hard hats and harnesses are small *and* tightly bound to a worker's body, so a few pixels of box-edge jitter under stochastic decoding represents a large fraction of

the box's own size, driving IoU down sharply even when the model is plainly still attending to roughly the same spot on the image.

Size alone, however, does not fully explain the pattern, and the data makes that point cleanly: **rule_3's guardrail and rule_4's excavator have almost identical median box area** — 221,163 px^2 versus 215,069 px^2, a difference of under 3% — **yet very different stability**, 0.615 against 0.922. If size were the whole story, these two rules should land at nearly the same `object_region_overlap_score`. They do not, and the remaining gap is best read as a **shape and rigidity** effect layered on top of size. A guardrail is long, thin, and sometimes visually segmented (formwork panels, intermittent rails), so small differences in where the model decides the structure's run starts or stops shift the box's extent noticeably between reruns, even when the area enclosed stays roughly similar. An excavator is one compact, rigid, visually distinctive object whose silhouette does not depend much on exactly where stochastic decoding happens to place the boundary — there is comparatively little ambiguity about where a single excavator begins and ends, which a long, segmented guardrail does not share. Size sets the floor on how much a few pixels of jitter can hurt IoU; shape decides how much jitter there actually is.

7.5 Worked Examples

Figure 7.1 shows two saved rerun overlays that make this chapter's size-and-shape finding visually concrete, one from each end of the `object_region_overlap_score` range.

(a) Image 23, rule 1 (hard hat): the overlaid top-1 (worker) boxes from all 3 reruns sit almost exactly on top of each other; the hard-hat box itself, not drawn here because it loses the area ranking to the worker box, is what actually moves (`object_region_overlap_score` = 0.069 for this sample).

(b) Image 79, rule 3 (guardrail): the guardrail/formwork box is nearly identical across all 3 reruns (`object_region_overlap_score` = 0.955).

Figure 7.1: Rerun overlays anchoring the hidden-instability finding: a stable worker detection masking an unstable hard-hat box (left) against a guardrail box that is stable in its own right (right).

Image 23 is the sharpest illustration of this chapter's central point. The three reruns' top-1 boxes — all worker boxes, since the worker is the larger object and wins the area ranking — overlay almost perfectly, which is exactly what feeds a healthy-looking `top_region_overlap_score`. But that overlay never draws the hard-hat box, because the hard hat loses the area ranking every time. The hard-hat

box's own overlap across the same three reruns, `object_region_overlap_score` = 0.069 for this specific sample, is far below even rule_1's already-low 0.499 mean — a near-complete miss for the object the rule is actually about, sitting directly underneath a top-1 score that looks fine. Image 79's guardrail box, by contrast, needs no such unmasking: it is the object actually being scored, and it sits nearly identically across all three reruns, `object_region_overlap_score` = 0.955, consistent with this chapter's account of large, rigid structures as the more stable case on both counts.

7.6 Standard vs. Non-Standard Choices

Standard: sampling-based stability and consistency testing — rerun the same input under controlled stochasticity and measure rerun-to-rerun agreement — is itself standard practice, rooted directly in stability as one of the six metrics defined for black-box XAI evaluation [3]. Switching to `do_sample=True`, `num_beams=1`, `temperature=0.7` specifically to avoid the vacuous deterministic-rerun case, and confirming non-vacuousness via `answer_agreement_rate` = 0.775 $\neq$ 1.0 before trusting the aggregate, follows that established methodology faithfully rather than improvising around it.

Non-standard: introducing `object_region_overlap_score` as a metric *separate from* the naive top-1-region IoU is this chapter's contribution. The six-metric framework's stability definition does not by itself specify which region a region-overlap score should be computed on when more than one candidate box is available per sample; the natural default — the top-ranked region from whatever heuristic produced the ranking — is exactly the choice that, here, hid a confound rather than revealing it. Computing a second, target-specific overlap score alongside the naive one, purpose-built to unmask the confound the naive metric was shown to hide, is a methodological refinement rather than a deviation from the framework, and is presented here as worth keeping in the framework going forward, not as a one-off patch for this pilot alone.

7.7 Implications for the Paper

The central methodological recommendation from this chapter is general, not specific to stability or to this pilot: whenever a region-overlap or region-based stability metric is computed on top of a ranking heuristic rather than on a metric's own true target object, the paper should report *both* numbers — the ranking heuristic's pick and the metric's own target object — rather than the ranking heuristic's pick alone. A single aggregate computed only on "whichever region a heuristic ranks first" can look acceptable while masking a target object that is substantially less stable, exactly as `top_region_overlap_score`'s 0.667 masked `object_region_overlap_score`'s 0.602 mean and, more sharply, rule_1's 0.499. This ties directly to the area-ranking limitation already flagged in Chapter 4[1]: any metric built on top of that ranking inherits its blind spot, and this chapter is the second of (at least) two places in the pilot where that inheritance produces a visible, quantified effect rather than a theoretical concern.

The same underlying pattern recurs here that Chapter 6 identified for visual sparsity, and the recurrence should be stated plainly rather than treated as coincidence: metrics computed on small, body-worn PPE items (hard hat, harness) come out systematically worse than the same metrics computed on large, rigid objects (excavator), for reasons that are at least partly mechanical — box size and shape interacting with IoU and with a 24×24-patch attention grid — rather than purely reflecting how good the underlying explanation actually is. Chapter 6 showed this

[1] Chapter 4's area-ranking heuristic, `standardize_regions`, ranks candidate boxes by pixel area as a stand-in for true importance, and that chapter's own implications section already forward-references this gap by name.

for attention concentration; this chapter shows it for rerun stability; **Chapter 11's cross-metric analysis is the natural place to bring the two findings together** rather than letting rule_1 and rule_2's weaker numbers be read in isolation, in either chapter, as evidence of worse explanations specifically for hard-hat and harness judgments. The irony is worth stating outright in the paper's discussion: hard hat and harness are exactly the PPE items for which a *stable* explanation matters most for real-world trust in a safety system, and they are the least stable by this measure, for reasons that trace back to object geometry rather than to any deficiency in how Florence-2 reasons about them.

Chapter 8

Efficiency and the Cost of Six Metrics at Scale (Day 8)

8.1 Motivation

Efficiency is one of the six metrics defined for evaluating black-box explainable-AI frameworks [3]: not whether an explanation is faithful or stable, but whether producing it is computationally affordable at the scale a real deployment or a larger study would require. Chapters 3 and 5–7 each ran Florence-2 inference for a different purpose — baseline judgments, masking-based reruns, cross-attention attribution, stochastic-decoding reruns — but none of them asked, in isolation, what the combined cost of running all four looks like, or whether that cost would still be reasonable at ten times this pilot's 163-sample size. Day 8's task was to answer exactly that question: aggregate the per-sample timing already implicit in the pilot's four inference-bearing days, extrapolate to a deployment-relevant scale, and report honestly on any gaps in the data needed to do so.

8.2 An Instrumentation Gap, Handled Honestly

The plan called for an `aggregate_timings()` routine to pull `inference_ms` straight out of Days 3, 5, 6, and 7's CSVs, with zero new model calls required. Checking the actual column headers first, rather than assuming the plan's premise held, showed that premise was only half true: of the four CSVs, only `baseline_predictions.csv` (Day 3) ever logged an `inference_ms` column at all. `descriptive_accuracy.csv` (Day 5), `visual_sparsity.csv` (Day 6), and `stability.csv` (Day 7) have no timing column whatsoever. Pulling pre-logged numbers from three of the four days was simply not available as an option, and the gap was handled differently depending on what each day's CSV could and could not support, rather than papered over with a single uniform assumption.

Day 5 required no new model calls. Day 5's descriptive-accuracy reruns use the *exact same call type* Day 3 already timed: `answer_rule()` at the default `num_beams=3`. Because the call type is identical, Day 5's cost can be recovered exactly from data already on disk, with no new inference run — as `rerun_count` × that sample's own Day 3 `inference_ms`, where `rerun_count` (1, or 2 if a second region existed for that sample) comes directly out of `region_extraction.csv`'s `n_regions` column. This is exact recovery, not estimation: every input to the computation is a real, already- measured quantity from this pilot's own data.

Days 6 and 7 required a small, bounded calibration. Day 6's cross-attention attribution and Day 7's stability reruns both use decoding configurations that nothing in the pilot had timed before — `output_attentions=True` (Day 6, forcing Florence-2's slower eager-attention path, as documented in Chapter 6) and `do_sample=True, num_beams=1` (Day 7, genuine stochastic sampling rather than beam search). Recovering their cost exactly from existing logs was not possible, because no existing log contains a timing for either configuration. Rather than invent a number or silently reuse Day 3's

unrelated timing, this script ran one small, fresh **15-sample calibration** for each configuration and scaled the result. This is a deliberate, bounded deviation from the plan's original "zero new model calls" premise, scoped the same disciplined way Chapter 6's own architecture deviation was scoped: small enough not to threaten Day 8's timebox, and documented here explicitly rather than folded silently into the results table as if it were exact.

8.3 Results

`scripts/08_efficiency_analysis.py` assembled Table 8.1 from the sources described in Section 8.2: exact aggregation for Days 3 and 5, calibrated extrapolation for Days 6 and 7. Day 4 (region extraction) and the cross-day analysis carried out in Chapter 11 perform no model inference at all — both are pandas/PIL operations over CSVs Days 3 and 6 already produced — and are correctly excluded from the table rather than padded in at zero cost.

Day	What	Calls/sample	Mean ms/sample	Extrap. to n =1,000
3	Baseline inference	1 `answer_rule` call (2 generate calls inside)	271	4.5 min
5	Descriptive-accuracy reruns	1.94 `answer_rule` calls	526	8.8 min
6	Cross-attention attribution (15-sample calibration)	1 `cross_attention_heatmap` call (1 generate call)	215	3.6 min
7	Stability sampled reruns	3 `answer_rule` calls	778	13.0 min

Table 8.1: Per-sample inference cost by day, on the 163-sample pilot, linearly extrapolated to $n = 1{,}000$.

Summed across all four rows, the **full six-metric pipeline at $n = 1{,}000$, by linear extrapolation, costs approximately 0.50 GPU-hours (about 30 minutes) on a single RTX 3070**. Practically trivial: efficiency is conclusively not a blocker for scaling this pilot to the full 3,004-image ConstructionSite test split, let alone to a 1,000-sample study.

8.3.1 Finding 1: comparing "ms per call" across days needs call-count normalizing

Read at face value, Table 8.1 appears to contradict Chapter 6's own finding that `output_attentions=True` forces a slower eager-attention path: Day 6's 215ms/sample looks *cheaper* than Day 3's 271ms/sample, not more expensive. The catch is a call-count mismatch hiding inside the per-sample average. `answer_rule()` (Days 3, 5, and 7) makes **two** `generate()` calls per logged "call" — one for the worker phrase, one for the safety-object phrase — while `cross_attention_heatmap()` (Day 6) makes only **one**. The two numbers are not measuring the same unit of work, and comparing them directly is comparing apples to (half-)oranges.

Normalizing to a single-`generate()`-call basis removes the mismatch. Day 3's per-call cost is its 271ms divided across its two internal generate calls, $271/2 \approx 135$ms; Day 7's 778ms/sample, divided across its three rerun calls (each itself two generate calls, so six generate calls per sample), gives the same per-rerun-call figure as Day 3's, ≈ 259ms per rerun call, implying $259/2 \approx 130$ms per generate call

— consistent with Day 3's 135ms, as it should be, since both use the same `num_beams=3`-vs-sampling decoding intensity per generate call. Day 6's single call, by contrast, costs the full 215ms on its own, with no division needed because there is only one generate call inside it. Normalized this way, Day 6's per-generate-call cost is **about 65% more expensive** than Day 3 or Day 7's per-generate-call cost (215 vs. ≈ 130–135ms) — consistent with the eager-attention overhead Chapter 6 already documented, once the comparison is made apples-to-apples. The lesson generalizes beyond this one pair of rows: any cross-day efficiency comparison in this pilot has to account for how many generate calls are bundled inside each logged "call" before treating raw per-sample milliseconds as comparable.

8.3.2 Finding 2: `num_beams=3` beam search is barely slower than `num_beams=1` sampling here

A second, independent finding falls out of the same table. Day 7's sampled calls (`num_beams=1`, ≈ 259ms per rerun call) are only about **4% faster** than Day 3's beam-search calls (`num_beams=3`, 271ms per call) — not the roughly 3× gap a naive "3 beams means 3× the decode compute" intuition would predict. The likely reason is that Florence-2's grounding outputs in this pilot are very short: location-token sequences of only about 7–8 generated tokens, per the token-count investigation already carried out in Chapter 6. With so few decode steps, each call's latency is dominated by the *fixed* cost of encoding the image through the DaViT vision tower once per call — a cost that does not depend on beam count at all — rather than by the decode loop, where beam multiplicity would actually matter. Beam count would likely matter far more for tasks that generate long sequences, such as `<MORE_DETAILED_CAPTION>`; it barely registers for this pilot's short grounding-token outputs.

8.4 Worked Example

Figure 8.1 breaks the headline per-sample numbers in Table 8.1 down visually, showing the relative cost of each day's inference workload behind the aggregate figures reported above.

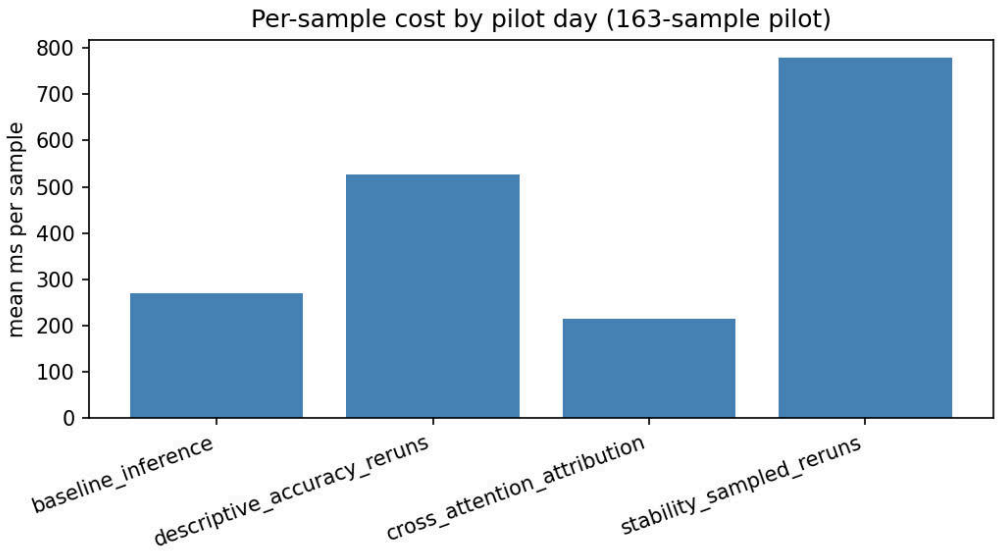


Figure 8.1: Per-sample inference cost by day (Days 3, 5, 6, 7), the timing breakdown behind Table 8.1's headline numbers and the $n = 1{,}000$ extrapolation.

8.5 Sanity Check on the Extrapolation

Because Days 6 and 7's per-sample costs rest on a 15-sample calibration rather than the full 163-sample run, the extrapolation those two rows feed into was checked against an independent order-of-magnitude estimate before being trusted. Day 3 and Day 5's numbers carry no such risk in the first place: they are exact sums over the real 163 samples (44.2s and 85.7s respectively), with no extrapolation step and therefore no extrapolation error possible. Day 6 and 7's numbers are a 15-sample calibration scaled linearly, first to the 163-sample pilot and then on to $n = 1{,}000$, and that scaling step is exactly where an error in the calibration would show up if one were present.

The calibrated 778ms/sample figure for Day 7 implies approximately **127 seconds** of pure GPU compute for all 163×3 sampled reruns. That figure is the right order of magnitude against what was actually observed running `07_stability_test.py` end-to-end: a few minutes of wall-clock time, with most of the gap between 127 seconds of compute and the longer observed wall-clock time attributable to one-time model loading and Hugging Face dataset streaming overhead, not to GPU compute. The calibration-based estimate and the direct observation are consistent, not off by an order of magnitude, which is the check this section set out to perform.

8.6 Standard vs. Non-Standard Choices

Standard: wall-clock timing aggregation and linear extrapolation for compute-budget estimation is itself standard practice, directly serving efficiency as one of the six metrics defined for black-box XAI evaluation [3]. Measuring per-sample inference time on a realistic pilot scale and extrapolating linearly to a deployment-relevant n is the conventional way to answer "is this affordable at scale," and this chapter follows that convention rather than departing from it.

Non-standard: the normalized-per-generate-call comparison introduced in Finding 1 is this chapter's own methodological contribution, not a step drawn from the six-metric framework as published. Comparing raw per-sample milliseconds across days that bundle different numbers of underlying `generate()` calls into a single logged "call" is an easy mistake to make and an easy one to miss — Day 6's 215ms genuinely does look cheaper than Day 3's 271ms until the call count inside each is accounted for, and the naive reading would have flatly contradicted Chapter 6's own eager-attention finding. Normalizing to a single-generate-call basis before drawing any cross-day conclusion is presented here as a detail worth stating explicitly in any paper's efficiency-methodology paragraph, not a one-off correction specific to this pilot.

8.7 Implications for the Paper

This chapter is the pilot's strong, low-controversy "good news" result. **Compute is conclusively not an obstacle to scaling this evaluation framework.** The full six-metric pipeline, extrapolated linearly from real, mostly-exact per-sample measurements, costs roughly half a GPU-hour at $n = 1{,}000$ on a single consumer-grade RTX 3070 — well within reach of scaling to the full 3,004-image ConstructionSite test split, and a result that should be reported in the paper without the qualifications that accompany several of this pilot's other findings. Where Chapters 6, 7, and others uncovered confounds, instability, or methodological deviations that temper how their numbers should be read, efficiency's central claim survives those caveats intact: even granting Day 6 and 7's calibration-based estimates the full uncertainty the sanity check in Section 8.5 affords them, the headline figure would have to be wrong by close to an order of magnitude before compute became a genuine scaling concern, and the sanity check found no such discrepancy.

That said, one caveat should be carried forward honestly into Chapters 11 and 12 rather than omitted for the sake of a clean headline: **Day 6 and 7's per-sample costs rest on a 15-sample calibration, not the full 163-sample run**, unlike Day 3 and 5's exact aggregation over all 163 samples. This is good enough for the order-of-magnitude story this chapter tells, and the sanity check in Section 8.5 found it consistent with direct observation, but it should be re-measured directly — by instrumenting `inference_ms` into Days 6 and 7's scripts themselves, the same way Day 3 already does — before quoting precise per-sample numbers in anything more formal than this pilot. The methodological recommendation worth carrying into the paper's own efficiency-methodology paragraph is Finding 1's normalization point: any reported per-sample or per-call timing for a multi-metric XAI pipeline should state explicitly how many underlying model-generate calls are bundled into that figure, since, as this chapter showed, failing to do so can make one method look artificially cheaper than another for reasons that have nothing to do with which method is actually faster.

Chapter 9

Robustness to Perturbation and Prompt Rewording: When a Stable Answer Hides a Drifted Explanation (Day 9)

9.1 Motivation

Robustness is one of the six metrics defined for evaluating black-box explainable-AI frameworks [3]: does a judgment, and the explanation behind it, survive small, realistic perturbations to the input, or does it depend brittlely on exact pixel values and exact wording? Chapters 3–8 each established a different facet of how Florence-2 behaves on a single, fixed input — its baseline answers, its grounded regions, its masking sensitivity, its attention concentration, its stochastic-decoding stability, its computational cost — but none of them perturbed the *input itself* and asked whether the answer and the explanation move together, move separately, or do not move at all. Day 9's task was to answer exactly that question, along two axes: realistic image perturbations (blur, lighting, occlusion, contrast) and prompt rewording (asking the same safety question in different words). The result is the richest chapter in this pilot, both in volume of findings and in the weight one of them carries: **a stable final answer can sit directly on top of an explanation that has completely drifted**, a result that reframes how every other chapter's numbers should be read.

9.2 Implementation

`perturbations.py` implements four Level-1 image perturbations and one stretch-test compositing operation. **`blur`** applies Gaussian blur with `ksize=8.0`. **`low_light`** applies gamma correction with `gamma=2.5`, darkening the image. **`occlude`** draws a centered black square covering `frac=0.2` of the image's area. **`contrast_shift`** scales pixel values around the mid-point by `factor=0.3`. **`paste_patch`** composites an arbitrary patch image onto an arbitrary box and is used only by the stretch test described in Section 9.9, not by the four Level-1 perturbations.

`robustness.py`'s `RobustnessResult` dataclass records four fields per rerun: `answer_changed` (did the final yes/no safety-rule answer flip), `confidence_drop` (baseline confidence minus perturbed confidence; a negative value means confidence *rose*), `object_box_iou` (intersection-over-union between the baseline's and the rerun's safety-object box, `NaN` if either side has no object box to compare), and `worker_lost` — a flag that fires specifically when the baseline run detected at least one worker but the perturbed rerun detected none. `worker_lost` exists because Chapter 5 already found that masking the only detected worker reroutes `_answer_presence_rule` into a degenerate scene-level fallback branch, flipping the answer for reasons disconnected from the safety concept actually being tested. The same risk applies to any perturbation that happens to make the worker undetectable, not only to a deliberate mask, so this chapter inherited Chapter 5's diagnostic rather than reinventing it.

`scripts/09_robustness_test.py` ran three test families over the 163-sample pilot. **Level 1**

reruns all 163 samples through each of the four image perturbations above, 652 reruns in total. **Level 2** reruns 138 of the 163 samples with the rule's second phrasing instead of an image change (`phrasing_index=1`); rule_4 is excluded, because `answer_rule` routes rule_4 to `_answer_rule_4`, which has no `phrasing_index` parameter at all — rule_4 is a fixed proximity pair (worker, excavator), not a phrase list, so passing `phrasing_index=1` for it would have silently been a no-op rerun of the exact same query, not a real test of wording sensitivity. This was caught before any aggregation code was written and rule_4's 25 excluded rows are logged with `excluded_reason="rule_4_has_no_second_phrasing"` rather than silently scored as 0% change, which would have understated rule_4's true (untested) rewording sensitivity by fabricating a result for it. **The stretch test** is a 20-sample synthetic hard-hat-patch experiment, covered separately in Section 9.9. All Level-1 and Level-2 reruns use the **same deterministic decoding as the logged baseline** (`num_beams=3`, no sampling), a deliberate methodological choice: without it, any answer change observed here could be attributed either to the perturbation or to Chapter 7's already-documented sampling noise, and there would be no way to tell which. Holding decoding fixed isolates perturbation sensitivity as the only thing that can change between baseline and rerun.

9.3 Architecture: A Perturbation Layer

Figure 9.1 lays out the three parallel branches `09_robustness_test.py` runs over each baseline sample: a Level-1 image-perturbation branch, a Level-2 reworded-prompt branch that leaves the image untouched, and a patch-stretch branch that composites a synthetic hard-hat patch onto the image before rerunning. All three branches converge on the same comparison step against the logged baseline, computing `answer_changed`, `confidence_drop`, `object_box_iou`, and `worker_lost` uniformly regardless of which branch produced the rerun.

Note that Level 2's branch is the only one of the three that leaves the image itself unperturbed — the input change is entirely in the text prompt, not the pixels — which is why it sits in the middle of the diagram with no image transformation drawn inside its top-row node, only the `phrasing_index=1` relabeling. The patch-stretch branch (right) is structurally a Level-1-style image perturbation but is kept visually and analytically separate because, unlike blur/low_light/occlude/contrast_shift, it targets one specific rule (rule_1) and one specific region (the approximate head box) by construction, rather than perturbing the whole frame uniformly.

9.4 Results

Table 9.1 reports the headline `answer_changed` rate for each of the five perturbation types, alongside the three diagnostic columns that distinguish *why* an answer changed from merely *whether* it changed.

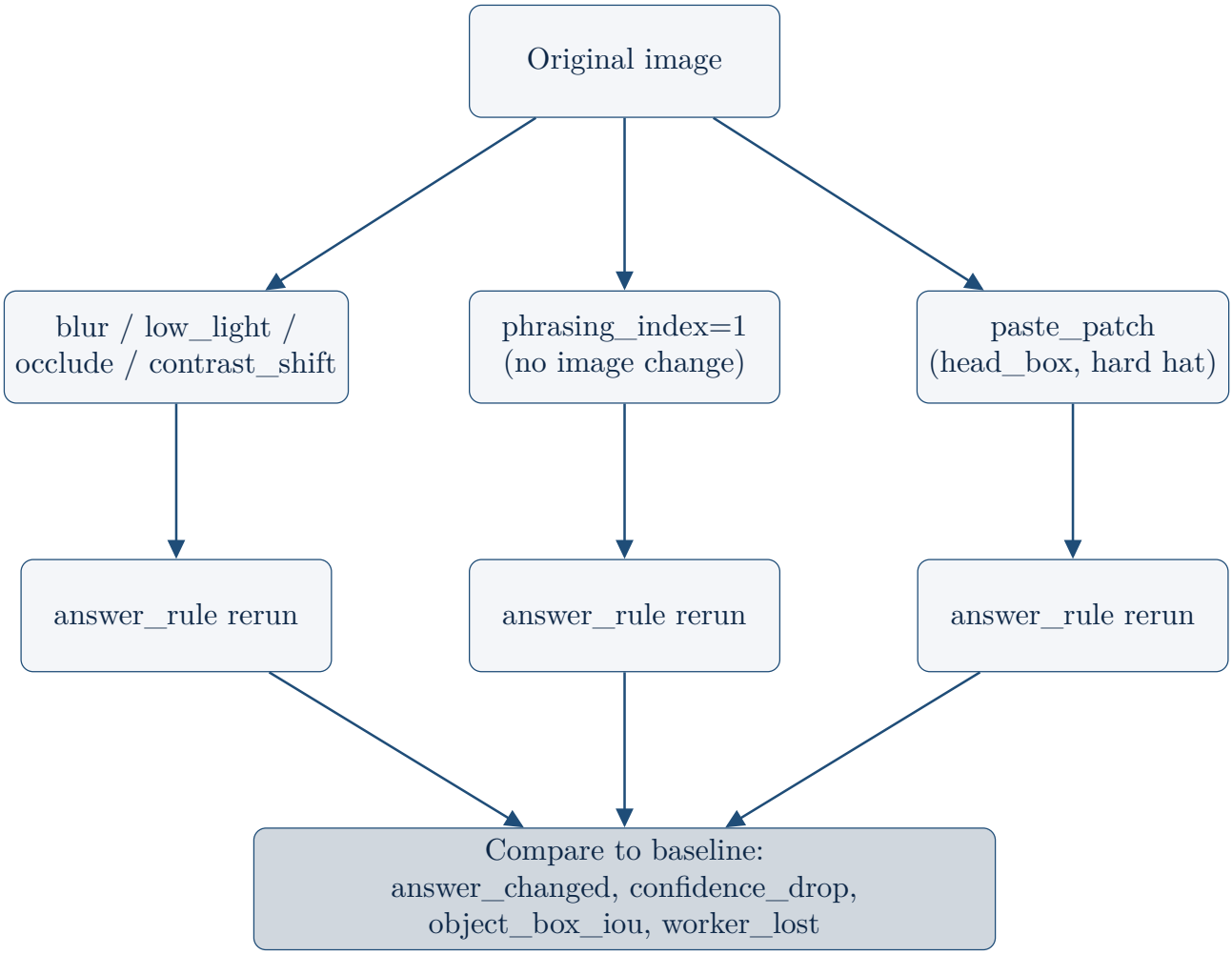


Figure 9.1: Perturbation layer: three parallel robustness branches, Level 1/Level 2/patch-stretch.

Perturbation	**answer_changed**	**worker_lost**	**mean confidence_drop**	**mean object_box_iou**
occlude	**28.2%**	8.6%	−0.012 (rose)	0.530
reworded_prompt	27.5%	0.0%	−0.053 (rose)	0.333
blur	27.0%	0.0%	0.013	0.287
low_light	11.7%	0.6%	0.011	0.816
contrast_shift	10.4%	0.0%	0.008	0.825

Table 9.1: `answer_changed` rate by perturbation, with the three diagnostic columns that explain *why*: `worker_lost` (the Chapter-5 fallback-rerouting risk), mean `confidence_drop` (negative means confidence rose), and mean `object_box_iou` (lower means the object box moved more).

Brightness and contrast changes are the mildest perturbations: object boxes barely move (`object_box_iou` above 0.8 for both) and answers rarely flip (10–12%). Blur, occlusion, and

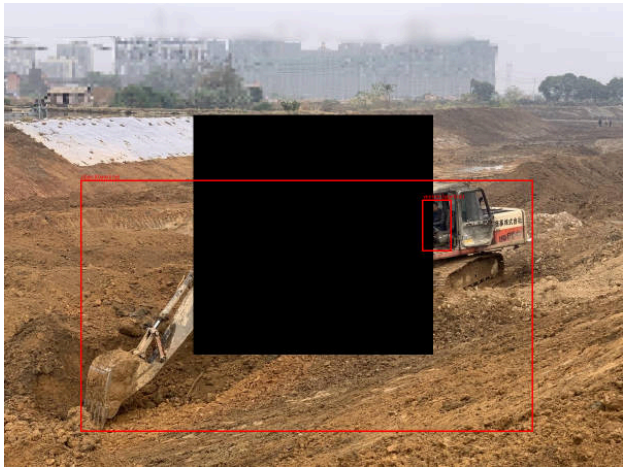
prompt rewording are all substantially more disruptive — answer-change rates between 27% and 28%, roughly 2.5× low_light and contrast_shift's rates — but, as Findings 1–4 below show in turn, **for three different reasons, not one**. Treating "blur, occlude, and reworded_prompt are all about equally disruptive" as a single finding would flatten three structurally different phenomena into one number; the rest of this chapter is about pulling them back apart.

9.5 Finding 1: Occlusion's Fallback Artifact

Occlude's 28.2% answer-change rate is not uniformly "genuine sensitivity to a covered region." **Of occlude's 46 answer-flip cases, 9 (20%) also have `worker_lost=True`** — the occluded rerun's fresh detection found no worker at all, routing `_answer_presence_rule` into its scene-level fallback branch for reasons disconnected from whether the safety object itself was actually still visible. The `worker_lost` rate under occlude is also far from uniform across rules: rule_2 19.2%, rule_4 16.0%, rule_3 8.2%, rule_1 1.6% — a more than tenfold spread between the highest and lowest rule, all under the identical occlusion transform.

Traced concretely on image `0000341` (rule_3, baseline "compliant"). Figure 9.2 shows the before/after pair. Before occlusion, **both baseline detections were already wrong**: the box labeled "worker" is actually a false positive on the excavator's cab window, and the box answering the guardrail query is the entire excavator silhouette, not a guardrail at all. Both boxes happened to satisfy the geometric coverage check the rule applies, producing a "compliant" answer that was correct only by accident. After occlusion, **both** detections vanish — `worker_lost=True`, `object_box_iou=NaN` — even though the occluded square does not fully cover either of the original (wrong) boxes. Occlusion is disrupting Florence-2's whole-scene grounding context, not merely blacking out the literal pixels the two boxes occupied. With no object box surviving the rerun, the fallback branch defaults to "violation" purely because no object box was found, flipping `answer_changed=True`.

(a) Before: both the "worker" box (a false positive on the excavator's cab window) and the guardrail-query box (the entire excavator silhouette) are wrong, but geometrically satisfy the coverage check.



(b) After occlude: both detections vanish (`worker_lost=True`, `object_box_iou=NaN`), even though the occluded square does not fully cover either original box.

Figure 9.2: Image `0000341`, rule 3: occlusion's fallback artifact, traced. The flip to "violation" reflects a degenerate fallback branch firing because no object box survived, not a correct detection that the guardrail became occluded.

Read in an aggregate table alone, this case is indistinguishable from "the model correctly noticed the

safety equipment became occluded." **It is not.** It is the same degenerate-fallback mechanism Chapter 5 already flagged for deliberate masking, here triggered by a perturbation instead. The two settings differ only in what disabled the detection — a hand-drawn mask there, a black occlusion square here — not in the underlying failure mode, which is that `_answer_presence_rule`'s fallback branch answers from the mere absence of any object box, with no way to distinguish "the safety object is genuinely gone" from "the detector lost its grip on the whole scene." Any future Bounded Completeness analysis (Chapter 10) needs to apply this same `worker_lost` check rather than reporting a raw `no_usable_explanation` rate blindly, since some fraction of "no usable explanation" cases will be this same fallback artifact rather than a genuine absence of grounding.

9.6 Finding 2: A Stable Answer Hiding a Drifted Explanation

This is the central result of Chapter 9, and arguably of the entire pilot. Quantified across all 611 stable-answer rows that have a comparable object box on both sides (baseline and rerun both detected at least one object box, and the answer itself did not change): **28.3% have `object_box_iou < 0.3`** — the answer stayed exactly the same, but the box the model points to as evidence moved to something close to unrelated. This is wildly uneven by perturbation, as Table 9.2 shows.

Perturbation	% of stable answers with `object_box_iou < 0.3`
blur	**58.8%**
reworded_prompt	56.7%
occlude	32.4%
low_light	5.7%
contrast_shift	2.8%

Table 9.2: Of samples whose final answer did not change, the percentage whose evidence box nonetheless drifted to `object_box_iou < 0.3`, by perturbation.

Under blur and reworded_prompt, **more than half** of all stable-answer samples are hiding a near-total relocation of the evidence box. This is not a tail effect or a rare edge case; it is the majority behavior for two of the five perturbation types.

Traced concretely on image `0000034` (rule_2, baseline "compliant", `answer_changed=False`). Figure 9.3 shows the before/after pair: the single best image in this pilot for demonstrating that answer-level robustness alone is not enough. The worker's entire baseline box sits **inside** the occluded square. A reasonable expectation would be that occluding the worker entirely either loses the detection (`worker_lost=True`) or, at minimum, forces the evidence box somewhere conspicuously different in a way that shows up as an obvious failure. Neither happens cleanly: the rerun still reports `worker_lost=False`, because it found *a* worker box somewhere in the image, and the final answer is unchanged — yet `object_box_iou=0.005`. The object box moved to a near-completely different location in the frame, while every answer-facing signal — the final yes/no answer, the `worker_lost` flag — reports business as usual.

An answer-level robustness number alone would call this sample perfectly robust; it is not, at the explanation level. This is the strongest single result in the pilot motivating the report of answer-level and explanation-level robustness as **two separate numbers, never blended into one**. A single blended robustness score computed only from `answer_changed` would report `0000034`

(a) Before: the worker's baseline box, fully visible.

(b) After occlude: the worker's entire baseline box sits inside the occluded square; the rerun still finds *a* worker box (`worker_lost=False`) and the answer is unchanged — but `object_box_iou=0.005`, an almost total relocation.

Figure 9.3: Image `0000034`, rule 2: **the centerpiece finding of this chapter.** An answer-level robustness number alone would call this sample perfectly robust. It is not, at the explanation level.

as a clean, unremarkable success — and would do so for 28.3% of all stable-answer rows across the dataset, systematically certifying as "robust" a population of samples whose explanations have, in fact, drifted to evidence the model is no longer actually looking at. This mirrors Chapter 6's decision to report visual and text attribution as two separate, never-merged streams: here, the two streams that must not be merged are what the model *answers* and what the model *points to as why*, and Finding 2 is the directly-traced, concrete failure case that motivates keeping them apart, rather than an assertion of that separation on principle alone.

9.7 Finding 3: Not All Phrasings Are Synonyms

Level-2 answer-change rate, broken down by rule, surfaces a pattern that initially looks like rewording sensitivity but is better read as a prompt-design issue specific to one rule:

Rule	reworded_prompt answer_changed
rule_2 (harness / lanyard)	**50.0%**
rule_1 (hard hat / vest)	31.7%
rule_3 (guardrail / edge barrier)	10.2%

rule_3's two phrasings ("guardrail" versus "edge protection barrier") are near-synonymous, and Florence-2 grounds them consistently — only a 10.2% answer-change rate. rule_2's two phrasings are **not** really synonyms in the way rule_3's are: a safety harness is a body garment worn by the worker, and a fall-protection lanyard is the tether attached to that garment. These are two different physical objects that happen to co-occur in the same PPE concept, not one object referred to by two interchangeable names. Asking Florence-2 to ground "harness" versus "lanyard" can legitimately produce two different

bounding boxes on the same scene without either grounding being wrong — the model is correctly answering two slightly different questions, not failing to recognize that one question was reworded. This is a prompt-design caveat for future revisions of `prompts.py`, not a model robustness failure: rule_2's two "phrasings" should probably be treated as two distinct concepts rather than as wording variants of a single concept, the next time `RULE_QUERIES` is revised.

9.8 Finding 4: An Uncalibrated Confidence Proxy

Mean `confidence_drop` is *negative* for both occlude (−0.012) and reworded_prompt (−0.053) — average confidence went **up**, not down, after the perturbation, despite both perturbations having among the highest answer-change rates in Table 9.1. If confidence behaved as a correctness signal, the perturbations that most disrupt the answer should be the ones that most erode confidence; instead, the two most disruptive perturbations are associated with the model growing *more* confident on average. This confirms, again, what Chapters 3 and 5 already cautioned: Florence-2's reported "confidence" is a mean token-generation-probability proxy, not a calibrated correctness signal. Removing visual information (occlusion) or changing the query wording can make the model surer of a different, not necessarily more correct, detection — confidence tracks how decisively the decoder committed to *some* answer, not whether that answer is right.

9.9 Stretch Test: A Synthetic Hard-Hat Patch

The stretch test pastes a synthetic hard-hat patch over the approximate head region of 20 `ppe_violation`/rule_1 candidates and asks whether the patch flips the answer from "violation" to "compliant" — a small, bounded probe of whether Florence-2's grounding can be spoofed by a crude, synthetic visual cue, scoped deliberately small per `technical-limitations-take-and-mitigation.md`'s guidance that a "Level 3" spoofing test should be a small part of this pilot, not a full promise of a demonstrated vulnerability.

The denominator-correction story. The naive headline number is a 15% (3/20) flip rate. That number was caught and corrected before being reported, because it silently mixes two populations that should not be combined. **16 of the 20 candidates already had a baseline proxy answer of "compliant,"** despite all 20 being dataset-labeled `ppe_violation` — a known sensitivity gap in the grounding-proxy method (the same class of issue Chapters 5 and 7 have already documented elsewhere), not something this patch test is designed to measure. For those 16 candidates, "fooling" the model into saying "compliant" is not a meaningful concept, because the proxy already says "compliant" before the patch is applied at all; counting them in the denominator dilutes the real signal with samples where the supposed "fooling" outcome was already the status quo. **Restricting to the 4 candidates the proxy already called "violation"** — the only 4 where a flip to "compliant" actually means something — **3 of 4 (75%) flipped to "compliant" after the patch was applied.** `09_robustness_test.py` now prints both numbers explicitly, the naive 15% (3/20) and the denominator-corrected 75% (3/4), rather than reporting only the naive figure.

$n = 4$ is far too small to generalize beyond "plausible, worth a larger run," and the four traced cases show no clean placement-to-flip correlation, summarized in Table 9.3.

Image	Patch placement	Flipped to "compliant"?
0000233	off-target (torso/shoulder, not head — a crude _head_box approximation, no pose estimation)	yes
0000641	on-target (squarely on the head)	no
0001431	on-target	yes
0001842	on-target, scene already has many real hard-hat wearers	yes

Table 9.3: The 4 patch-stretch candidates for which a flip is a meaningful outcome (baseline proxy answer already "violation"), with patch placement traced individually against the flip outcome.

Figure 9.4 shows the two cases that make the placement question concrete: a well-placed patch (0000641) that failed to fool the model, against an off-target patch (0000233) that succeeded.

(a) 0000233 before: baseline "violation."

(b) 0000233 after: patch placed off-target (torso/shoulder, not head) — flipped to "compliant" anyway.

(c) 0000641 before: baseline "violation."

(d) 0000641 after: patch placed squarely on-target (the head) — did *not* flip.

Figure 9.4: The counterintuitive on-target/off-target pair: a well-placed patch (bottom) failed to fool the model, while an off-target patch (top) succeeded.

A well-placed patch failed to fool the model while an off-target one succeeded — the flip does not cleanly

track anatomically correct placement in either direction. Florence-2's "hard hat" grounding appears generally imprecise about *where* on a person the visual cue needs to be, sometimes too lenient (an off-target patch still registers as "compliant") and sometimes not lenient enough (an on-target patch on `0000641` did not). This is exactly the kind of result `technical-limitations-take-and-mitigation.md` anticipated for a bounded stretch task: a real, traceable signal, explicitly not strong enough to claim a demonstrated real-world spoofing vulnerability from $n = 4$.

9.10 Standard vs. Non-Standard Choices

Standard: perturbation-based robustness testing — blur, noise, occlusion, and related image transformations rerun through the same model and compared against a baseline — is itself standard practice, the same family of method behind robustness as one of the six metrics defined for black-box XAI evaluation [3]. Holding decoding deterministic and identical to the logged baseline across all reruns, specifically to avoid confounding perturbation sensitivity with Chapter 7's already-documented sampling-noise sensitivity, follows that established practice rather than departing from it.

Non-standard: the explicit separation of answer-level and explanation-level robustness into **two never-blended numbers** is this chapter's central methodological contribution, motivated directly by Finding 2's traced concrete failure case (image `0000034`) rather than asserted on principle alone. The six-metric framework's robustness definition does not by itself mandate this split; the natural default is a single `answer_changed` rate, and that default is exactly what would have certified 28.3% of stable-answer samples as fully robust despite a near-total evidence-box relocation underneath. The denominator-correction in the patch-stretch test (restricting to the 4 candidates for which a flip is a meaningful outcome, rather than reporting the naive 15% over all 20) is this chapter's second non-standard, reportable choice: catching a vacuous-denominator issue before publishing a headline number, rather than after.

9.11 Implications for the Paper

Findings 1 and 2, taken together, make the strongest single argument in this pilot for the paper's central thesis that answer-level metrics are necessary but not sufficient for VLM explainability. Finding 1 shows that an answer-level metric can flip for a reason that has nothing to do with the safety concept being tested (a degenerate fallback branch, not genuine sensitivity to the occluded region). Finding 2 shows the complementary failure: an answer-level metric can stay *perfectly stable* while the explanation underneath it relocates almost completely. Read side by side, the two findings close off both directions an answer-only evaluation could fail in — an answer that changes for the wrong reason, and an answer that does not change despite the explanation collapsing — and neither failure mode is visible from `answer_changed` alone in either case. **This chapter's two traced case studies, `0000341` (Finding 1) and `0000034` (Finding 2), are the best candidates in the entire pilot for the paper's qualitative-analysis figure**: both are visually self-explanatory before/after pairs that make an abstract methodological point concrete in a single glance, without requiring the reader to parse an aggregate table first.

The answer-level-versus-explanation-level split established here recurs as this pilot's central cross-chapter theme: Chapter 10's Bounded Completeness analysis, Chapter 11's cross-metric rollup, and Chapter 14's open questions for Professor Abdallah all return to this same distinction, and this chapter is where it was first motivated by a directly-traced failure rather than introduced as an abstract design choice. Three secondary points should also carry forward. **First**, Finding 3's caution that rule_2's two phrasings ground genuinely different physical objects (a harness and a lanyard), not two wordings

of one concept, should inform any future revision of `prompts.py` before rule_2's rewording rate is cited as evidence of model instability. **Second**, Finding 4's confirmation that the confidence proxy is uncalibrated — rising, not falling, under exactly the two most disruptive perturbations — reinforces the standing recommendation, carried since Chapter 3, that this pilot's "confidence" figure never be reported as a correctness or trust signal in the paper without that caveat attached explicitly. **Third**, the patch-stretch test's denominator-correction is worth stating as a general methodological note in the paper's own methods section: any spoofing- or adversarial-style probe of a binary safety classifier must first verify what fraction of its candidate set already sits on the target side of the decision before computing a "fooling rate," or risk silently diluting a real signal with samples for which the test's notion of "fooling" was never well-defined to begin with.

Chapter 10

Bounded Completeness: Is the Top Region Actually Necessary? (Day 10)

10.1 Motivation

Completeness is one of the six metrics defined for evaluating black-box explainable-AI frameworks [3]: does the explanation a model offers actually account for its decision, or does the decision survive even when the explanation is taken away? A full completeness analysis would ablate the entire feature space and ask whether *any* subset of removed evidence changes the output — a combinatorially large undertaking well outside this pilot's scope. Day 10 instead implements a deliberately **bounded** variant, scoped down to the one region the pipeline already treats as most important. Per `technical-limitations-take-and-mitigation.md`:

> *"A sample passes the bounded completeness check if the model provides a usable explanation and if perturbing the top-ranked region or top two regions causes a measurable change in the model's answer, confidence proxy, or grounding output."*

This is a necessity test, not a sufficiency test: it asks whether the single region Chapter 4's `standardize_regions` ranked first is actually load-bearing for the model's answer, or whether the model would have reached the same conclusion regardless of whether that region existed at all. Chapters 4 and 5 already built every piece this chapter needs — ranked regions and the masked-rerun answer-change signal — so Day 10's task is substantially a rollup of work already done, plus one narrowly bounded extra rerun set to check a risk the rollup alone cannot resolve.

10.2 Implementation

`completeness.py`'s `classify_sample()` is a pure rollup over two existing columns: Day 4's `region_extraction.csv` (`top_region_source`, which records whether a real model-returned box or only the grid fallback was available) and Day 5's `descriptive_accuracy.csv` (`answer_changed_top1` and `answer_changed_top2`). No new model calls are required for the headline classification: a sample with no model-returned region at all is `no_usable_explanation`; a sample with a model region whose masking changed the answer (top-1 or top-2) is `explanation_supported`; a sample with a model region whose masking did *not* change the answer is `explanation_weak`. `scripts/10_bounded_completeness.py` ran this rollup over all 163 pilot samples end-to-end and wrote `results/bounded_completeness.csv` (163 rows).

One extra, explicitly bounded set of fresh model calls was made on top of the rollup: **159 top-1-only mask reruns**, reusing Chapter 4's already-extracted top-1 box rather than re-ranking from scratch,

that additionally capture post-mask `worker_boxes`. This was necessary because Chapter 5's own CSV never logged post-mask boxes, only the answer-change outcome; the rollup alone cannot tell whether an answer change reflects the top region genuinely mattering or reflects the same worker-detection-fallback-rerouting artifact that Chapters 5 and 9 already documented being triggered by other means. Resolving that question — the substance of Finding 2 below — required this one additional, narrowly scoped rerun set, not a new pipeline stage.

Why no confidence-proxy threshold. The bounded definition explicitly permits a measurable change in the confidence proxy as an alternative passing signal alongside answer change. This was checked before being ruled out: among Chapter 5's `answer_changed_top1 == False` rows, the distribution of `abs(confidence_drop_top1)` (median 0.032, 75th percentile 0.057) is essentially indistinguishable from the same statistic over the full population (median 0.032, 75th percentile 0.057). Confidence drop carries no discriminative signal between "answer changed" and "answer did not change" at this scale; adding a threshold on top of it would inject noise into the classification rather than recover real signal. `classify_sample()` therefore uses only the categorical answer-change signal, consistent with Chapters 3 and 9's standing caution that this pilot's confidence figure is an uncalibrated proxy, not a correctness signal.

10.3 Results

Table 10.1 reports the headline classification across all 163 samples.

Verdict	n	%
`explanation_weak`	88	54.0%
`explanation_supported`	71	43.6%
`no_usable_explanation`	4	2.5%

Table 10.1: Headline bounded-completeness classification, all 163 pilot samples.

A slim majority of samples (54.0%) land in `explanation_weak`: the top-ranked region exists and is masked, but the answer does not move. Read uncritically, that could be mistaken for "the model is mostly ignoring its own top-ranked evidence" — Findings 1 and 2 below show that this headline number is itself shaped by a fixable mechanism, not an inherent property of the model.

Hard subsets are reported separately here, by deliberate policy, never blended into the overall rate, exactly as the plan specifically asks. Table 10.2 reports two such subsets alongside the overall figure for comparison.

Subset	n	supported	weak	no_usable
overall	163	43.6%	54.0%	2.5%
`quality_of_info == "poor info"`	84 (51.5% of pilot)	39.3%	58.3%	2.4%
multi-rule-violation (n=7, too small for conclusions)	7	42.9%	57.1%	0.0%

Table 10.2: Hard subsets reported separately from the overall rate, by deliberate policy, never blended into the headline figure.

The poor-info subset — just over half the pilot — is mildly worse than the overall rate (39.3% vs. 43.6% supported) but not dramatically so; whatever is driving `explanation_weak`'s headline share is not concentrated in the samples with poor underlying information quality. The multi-rule-violation subset has only 7 samples, too few to support any real conclusion; it happens to land close to the overall rate, but with $n = 7$ that proximity is not itself evidence of anything.

10.4 Finding 1: The Area-Ranking Mechanism, Again

The headline 54.0% `explanation_weak` rate is not uniform across rules, and the pattern it follows is the same mechanism Chapter 4 flagged as a foreseen limitation back when `standardize_regions` was first designed: ranking candidate regions by box area, descending, rather than by relevance to the rule being tested. Table 10.3 breaks the verdict down by rule.

Rule	supported	weak	no_ usable	n
rule_1 (hard hat / vest)	**33.3%**	66.7%	0.0%	63
rule_2 (harness / lanyard)	46.2%	50.0%	3.8%	26
rule_3 (guardrail / edge barrier)	**55.1%**	38.8%	6.1%	49
rule_4 (excavator proximity)	44.0%	56.0%	0.0%	25

Table 10.3: Bounded-completeness verdict by rule. rule_1 is the weakest; rule_3 is the strongest — a spread of nearly 22 percentage points in `explanation_supported`.

rule_1 (hard hat/vest, the PPE rule the pilot cares about most directly) supports the model's answer in only 33.3% of cases; rule_3 (guardrail/edge barrier) supports it in 55.1% — a 21.8 percentage point spread between the weakest and strongest rule. This is not noise: it traces to exactly which entity `standardize_regions` ranks top-1 for each rule. For rule_1, the worker's whole-body box is almost always larger than the hard-hat or vest box it is being compared against, so the area-ranking heuristic promotes the **worker** to top-1 in **48/50 (96%)** of `ppe_violation`/ rule_1 samples. For rule_3, the guardrail itself is frequently a large structural object spanning much of the frame, so the **guardrail** is ranked top-1 in **33/50 (66%)** of `fall_hazard`/rule_3 samples, with the worker ranked top-1 in only 12/50 (24%) of those same samples.

Collapsing this across all 163 samples by which entity type was ranked top-1, rather than by rule, makes the mechanism explicit, in Table 10.4.

Top-1 region is. . .	supported	weak	n
the worker	**37.5%**	62.5%	80
the safety/hazard object	**49.4%**	45.8%	83

Table 10.4: `explanation_supported` rate by which entity type was ranked top-1. (Row percentages for the safety/hazard-object row sum to 95.2%; the remainder is `no_usable_explanation`.)

Masking the worker is reliably less likely to flip the answer than masking the actual safety or hazard object — 37.5% versus 49.4%, an 11.9 percentage point gap that holds across the whole pilot, not just within one rule. **This is a real, fixable limitation of the ranking heuristic, not a property of the model being explained.** Area is a reasonable proxy for visual salience in the absence of any true attention or saliency signal from Florence-2 (the same constraint Chapter 4 flagged at the point the heuristic was designed), but it systematically promotes the wrong entity to top-1 exactly when the safety-relevant cue is small relative to the person wearing or standing near it — which is precisely the PPE case (rule_1) this pilot, and the eventual paper, cares about most. A ranking heuristic that instead ranked by "does this region match the rule's queried object class" before falling back to area would directly correct the asymmetry visible in Table 10.4.

10.5 Finding 2: A Worker-Fallback Artifact Inflates the Headline Number

The 159 top-1-only mask reruns described in the Implementation section were run specifically to check whether the worker-detection-fallback-rerouting risk — flagged in Chapter 5 and quantified under perturbation in Chapter 9 — also contaminates this chapter's own `explanation_supported` count. It does, in a small but precisely quantifiable slice.

Of the 159 model-region samples, masking the top-1 region caused the rerun to lose its only detected worker entirely (no worker redetected post-mask) in **12 cases (7.5%)**. Of the 71 samples classified `explanation_supported`, **3 (4.2%)** co-occur with this artifact: their "supported" verdict is plausibly driven by `_answer_presence_rule`'s scene-level fallback branch (`"compliant" if object_boxes else "violation"`) firing because no worker survived the mask, rather than by the top region being genuinely necessary to the rule's actual reasoning. A corrected, "genuinely-supported" count would be **68/163 (41.7%)** rather than the naive **71/163 (43.6%)** reported as the headline figure above — a small but real overstatement that would have gone unflagged without this extra rerun set.

Traced concretely on image `0000642` (rule_2, baseline "compliant"). Figure 10.1 shows the before/after pair. The model's "worker" box `(436.2, 232.2, 499.8, 419.6)` and its "harness" box `(438.6, 232.2, 495.0, 419.6)` are almost pixel-identical — Florence-2 returned nearly the same region for both queries on this image. Masking the top-ranked "worker" region therefore also erases the harness evidence in the same stroke, visible in the after panel: the black box swallows the only worker-relevant region in frame. The rerun fails to redetect any worker, triggers the fallback branch, and the answer changes — registering as `explanation_supported`, when what actually happened is the same degenerate-fallback mechanism first flagged in Chapter 5 and quantified under image perturbation in Chapter 9, here triggered by this chapter's own masking procedure rather than by noise or a deliberate perturbation.

Read in an aggregate table alone, `0000642` is indistinguishable from a genuine necessity result — masking the top region changed the answer, exactly what `explanation_supported` is supposed to mean. **It is not.** It is the same fallback mechanism that has now surfaced under three different triggers across this pilot: deliberate masking (Chapter 5), realistic perturbation (Chapter 9), and this chapter's own bounded-completeness mask reruns. Any completeness-style metric built on top of a model with conditional or fallback logic needs to check for this kind of artifact explicitly, the same way this chapter did with its extra 159-rerun set, rather than trusting a raw answer-change rate to mean what it appears to mean.

(a) Before: the "worker" box and the "harness" box are almost pixel-identical — Florence-2 returned nearly the same region for both queries.

(b) After masking top-1 ("worker"): the harness evidence is erased in the same stroke; the rerun finds no worker, the fallback branch fires, and the answer changes.

Figure 10.1: Image `0000642`, rule 2: the worker-fallback artifact, traced. The "supported" verdict this case earns reflects a degenerate fallback branch firing on a lost detection, not the top region genuinely being necessary to the rule's reasoning.

10.6 Finding 3: Grid-Fallback Is Rare but Concentrated in rule_3

Only **4/163 samples (2.5%)** had no native grounding box at all and were classified `no_usable_explanation`. Table 10.5 lists all four.

Image ID	Primary class	Rule	Quality of info
0000253	fall_hazard	rule_3	rich info
0000361	fall_hazard	rule_3	poor info
0000620	compliant	rule_2	rich info
0001229	fall_hazard	rule_3	poor info

Table 10.5: All four samples classified `no_usable_explanation` (no native grounding box returned at all).

Three of the four are rule_3. "Guardrail" or "edge protection barrier" is a harder open-vocabulary detection target for Florence-2 than "hard hat" or "worker," consistent with rule_3 already showing the lowest grounding hit-rate signal back in Chapters 3 and 4. The rate is small enough that it barely moves the headline number, but the concentration in a single rule is consistent rather than random, and is worth noting alongside Finding 1's rule_3 result: rule_3 sits at both extremes in this chapter, the strongest rule by `explanation_supported` rate when a grounding box *is* returned, and the rule most likely to return no grounding box at all.

10.7 Standard vs. Non-Standard Choices

Standard: completeness, or necessity testing via top-k region ablation, is itself standard practice, directly the same metric defined for black-box XAI evaluation [3]. The "bounded" scoping used here —

restricting the ablation to the top-ranked region or top two regions, rather than the full feature space — is an explicit, acknowledged narrowing of that standard definition, adopted because a full completeness sweep over every combination of regions was out of scope for a pilot of this size, not a departure from the metric's underlying logic.

Non-standard: two choices in this chapter are this pilot's own contribution rather than inherited from the six-metric framework. First, the explicit policy of reporting hard subsets (poor-info samples, multi-rule-violation samples) **separately, never blended into the overall rate** — Table 10.2 rather than a single pooled number — guards against a harder subset's worse performance being diluted into an artificially reassuring headline figure, or conversely against a too-small subset ($n = 7$) being mistaken for a real signal. Second, and more significant: quantifying *exactly how much* the fallback-rerouting artifact inflates the headline number (Finding 2's 68/163 vs. 71/163 correction) via a dedicated extra rerun set, rather than reporting the naive rollup number and trusting it. This is a rigor practice worth recommending generally for any completeness-style metric built on top of a model that has conditional or fallback logic anywhere in its answer pipeline: the metric's own perturbation procedure can itself trigger the same fallback paths that contaminate the underlying model's answers, and a rollup alone cannot detect that without an extra, targeted check like the one performed here.

10.8 Implications for the Paper

Findings 1 and 2, taken together, make the strongest single case in this pilot for recommending that the region-ranking heuristic be fixed before this pipeline is scaled up to a larger dataset. Finding 1 shows that the heuristic introduced in Chapter 4 — rank candidate regions by box area, descending — produces a structural, rule-dependent bias: it promotes the worker to top-1 for PPE rules (96% of rule_1 samples) precisely because the worker's box is almost always larger than the PPE item itself, and masking the worker is measurably less likely to flip the answer (37.5% supported) than masking the actual safety object (49.4% supported). That gap is not a property of Florence-2's underlying competence; it is an artifact of which region the heuristic happens to rank first, and it depresses the bounded-completeness rate hardest exactly on rule_1, the PPE rule this pilot and the eventual paper care about most directly. Finding 2 shows that the same underlying fallback mechanism this pilot has now traced under three different triggers (Chapter 5's deliberate masking, Chapter 9's realistic perturbations, and this chapter's own bounded mask reruns) also inflates the headline `explanation_supported` count by a small but precisely quantified amount (3/71, correcting 43.6% down to 41.7%). Read together, the two findings say the same thing from two directions: **the area-ranking heuristic is the single highest-leverage, most fixable problem surfaced anywhere in this pilot**, because it is not a limitation of the underlying model or the evaluation metric's design, but of one specific, identified, and replaceable function (`standardize_regions`) that every later masking-based chapter in this pilot — Chapters 5, 6, 7, and this one — inherits without modification.

This recommendation is not new to this chapter; Chapter 4 forward-referenced it at the point the heuristic was first designed, anticipating that the area-based choice would resurface as a limitation downstream. **This chapter is where that anticipated limitation is finally measured directly and attributed to a specific, fixable cause**, and it should be presented as the paper's top recommended next engineering step, tied directly to Chapter 14's task list item 3: replace `standardize_regions`'s area-only ranking with a rule-aware ranking that prefers a region matching the rule's queried object class (hard hat, vest, harness, lanyard, guardrail, excavator) before falling back to area only when no class match is available. Such a fix would not require any new model calls or architectural change — only a re-ranking of boxes Florence-2 has already returned — and Finding 1's Table 10.4 gives a direct,

falsifiable prediction for what a successful fix should produce: the 11.9 percentage point gap between masking the worker and masking the safety object should narrow substantially, if not close outright, once the ranking stops systematically promoting the wrong entity to top-1 for PPE rules.

Two secondary points should also carry forward. **First**, Finding 3's rule_3 concentration in `no_usable_explanation` is a second, independent piece of evidence (alongside Chapters 3 and 4's already-noted grounding hit-rate signal) that "guardrail"/"edge protection barrier" is a harder open-vocabulary target for Florence-2 than the other rules' queried objects, worth flagging if rule_3 is scaled up without also revisiting its grounding prompt. **Second**, the Standard vs. Non-Standard section's rigor practice — explicitly checking whether a metric's own perturbation procedure can trigger the same fallback paths that contaminate the underlying model, rather than assuming a clean separation between "the metric" and "the model under test" — generalizes beyond completeness and is worth stating in the paper's methods section as a general caution for evaluating any model with conditional or fallback logic in its decision pipeline.

Chapter 11

Aggregating Six Metrics into a Single Cross-Class Picture (Day 11)

11.1 Motivation

Chapters 5–10 each implemented one metric from the six-metric framework for evaluating black-box explainable-AI systems [3] — descriptive accuracy, visual sparsity, stability, robustness, efficiency, and bounded completeness — and reported each one's results in isolation, against its own headline number, its own by-rule breakdown, and its own traced failure cases. None of those six chapters asked the cross-metric question directly: *read side by side, broken down by the same four hazard classes, what picture do all six metrics paint together?* Day 11's task was exactly that rollup — and, by deliberate design, nothing more. **No new model calls and no recomputed logic are introduced in this chapter.** Every number that follows is a `groupby(primary_class).mean()` over a column Chapters 5–10 already computed and already verified; `scripts/11_make_figures.py` reads `descriptive_accuracy.csv`, `visual_sparsity.csv`, `stability.csv`, `robustness.csv`, `bounded_completeness.csv`, and `efficiency_summary.csv` and writes one summary table, `results/pilot_metric_summary.csv` (six rows, one per metric), and one chart, `results/figures/summary/metric_summary_by_class.png`.

This chapter's contribution is not new measurement; it is the discipline of *reading the six metrics together without overstating what that joint reading is allowed to mean.* Two of the four findings below exist specifically to guard against a natural but mistaken cross-metric inference: that two metrics moving together by class is independent confirmation of one underlying story (Finding 1 shows it sometimes is not, by construction), and that a metric measuring "focused attention" must agree with a metric measuring "decision-critical region" (Finding 2 shows it need not). One further tooling note belongs here because Day 11 is where the pilot's visual-review practice is named explicitly for the first time: throughout this pilot, every saved figure — the overlays in Chapter 3, the masked-image pairs in Chapter 4, the heatmaps in Chapter 6, the before/after pairs in Chapters 9 and 10, and this chapter's own summary chart — was visually reviewed by reading the saved PNG directly (Claude Code's `Read` tool rendering the image), not by opening a Jupyter notebook. The `notebooks/` directory scaffolded at the start of the pilot was never actually used.

11.2 Architecture: The Complete Pipeline

Figure 11.1 is the capstone diagram of this report: the complete eleven-script pipeline as it stands after Day 11, with every earlier chapter's architecture diagram visible inside it as a sub-piece rather than superseded by it. Scripts `01`–`04` are Diagram A (Chapter 1's environment-and-inference path) extended by Diagram B (Chapter 3's person-relative baseline redesign, folded into script `03`) and Diagram C (Chapter 4's region-standardization and masking layer, script `04`). From script `04`'s ranked regions, the pipeline fans out into five parallel metric scripts, `05`–`09`: `05` (descriptive accuracy, Chapter 5),

`06` (visual sparsity, Chapter 6, which is Diagram D's decoder→encoder cross-attention layer), `07` (stability, Chapter 7), `08` (efficiency, Chapter 8), and `09` (robustness, Chapter 9, which is Diagram E's three-branch perturbation layer). Script `10` (bounded completeness, Chapter 10) sits downstream of all five parallel metric scripts because it is itself a rollup over two of them (`05`'s answer-change columns and `04`'s region-source column), not a sixth independent measurement running in parallel. Script `11` — this chapter — is the final node, collecting all five metric CSVs plus `10`'s completeness CSV into the one summary table and one chart reported below. **Diagram F is, in this precise sense, the composition of Diagrams A, B, C, D, and E from Chapters 1, 3, 4, 6, and 9**: nothing in it is a new architectural component, only the assembled shape of components each already introduced and individually verified in those five chapters.

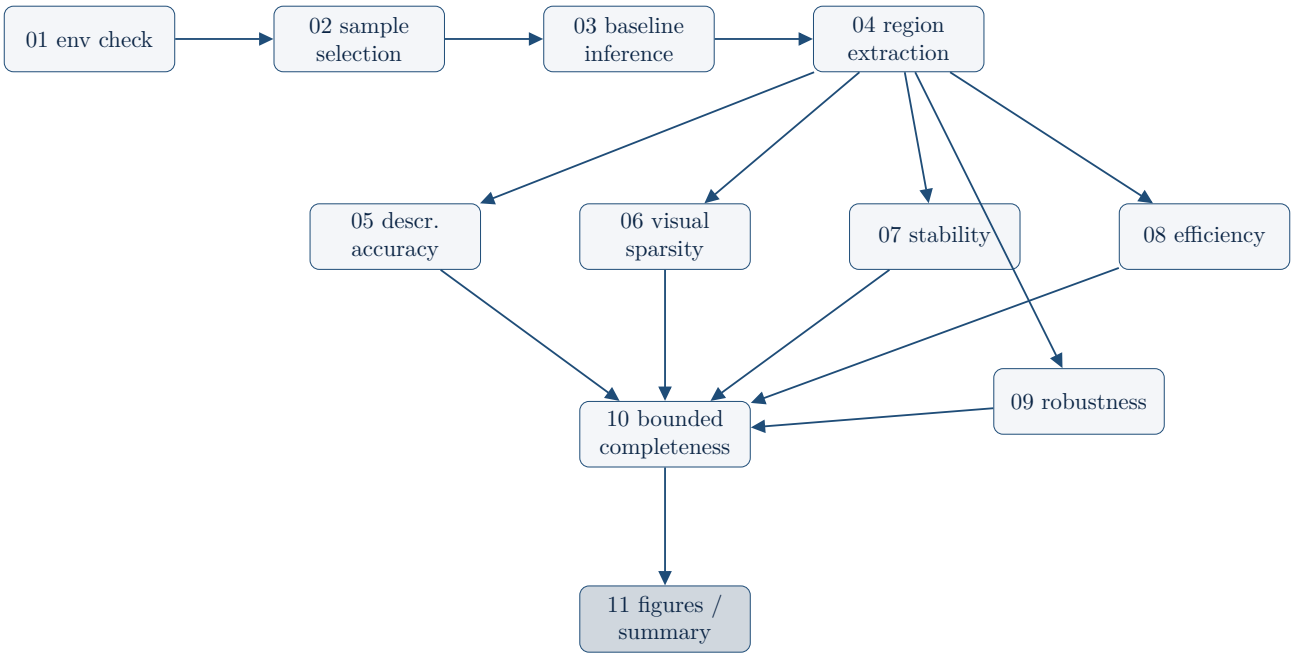


Figure 11.1: The complete 11-script pipeline after Day 11 — the composition of Diagrams A-E.

11.3 Results

Table 11.1 reproduces `pilot_metric_summary.csv` in full: all six metrics, their overall value, and their per-class breakdown across the pilot's four hazard classes.

Metric	Overall	compliant	ppe_violation	fall_hazard	struck_by_risk (n=13)
descriptive_accuracy	36.2%	28.0%	28.0%	46.0%	**61.5%**
visual_sparsity_top5	0.108	0.099	0.127	0.109	**0.067**
stability	77.5%	74.7%	76.0%	78.7%	**89.7%**
robustness Level 1	80.7%	79.5%	86.0%	**75.0%**	86.5%
bounded_completeness	43.6%	32.0%	36.0%	58.0%	**61.5%**
efficiency (n=1,000, aggregate)	~**0.50 GPU-hours** (single RTX 3070; aggregate, not per-class)				

Table 11.1: The headline cross-metric summary, reproduced from `pilot_metric_summary.csv`. Bold marks each row's strongest class (efficiency has no per-class breakdown: it is an aggregate runtime budget, not a per-sample classification). **struck_by_risk rests on n=13, not n=50** — the balanced 50-per-class sampler could not find 50 struck_by_risk images in the 3,004-image test split, a constraint already noted in Chapter 2. Every struck_by_risk number in this table should be read as a directional signal, not a stable estimate.

11.4 Worked Example

Figure 11.2 is the single most important figure in this report: the one chart that places all five per-sample metrics (efficiency is excluded, as it has no per-class or per-sample breakdown to plot) side by side, grouped by hazard class, on their own native scales. It is the visual form of Table 11.1 and the source the four findings below were read directly from — each bar height was cross-checked against the printed table before any finding was drafted, rather than trusting the chart's proportions alone.

11.5 Finding 1: descriptive_accuracy and bounded_completeness move together by class — but that's expected, not independent confirmation

descriptive_accuracy and bounded_completeness trace nearly the same shape across the four classes: both are weakest for compliant (28.0% / 32.0%) and ppe_violation (28.0% / 36.0%), and both are strongest for fall_hazard (46.0% / 58.0%) and struck_by_risk (61.5% / 61.5%). Read carelessly, two metrics agreeing this closely could be mistaken for two independent lines of evidence converging on the same conclusion. **They are not independent.** Chapter 10's `bounded_completeness` classification is *built from* Chapter 5's own `answer_changed_top1` and `answer_changed_top2` columns by construction — `classify_sample()` labels a sample `explanation_supported` precisely when one of those two columns is `True`. The two metrics sharing a shape across classes is therefore close to a methodological identity, not a second, independently-arrived-at confirmation of the same underlying pattern.

The real, independent explanation for *why* both numbers are low for compliant/ ppe_violation and high for fall_hazard/struck_by_risk was already traced in Chapter 10, Finding 1, not discovered here: `standardize_regions`'s area-based ranking promotes the worker's whole-body box over the smaller hard-hat or vest box for rule_1, and masking the worker is measurably less likely to flip the proxy's answer than masking the actual safety object (37.5% versus 49.4% supported, Chapter 10). This chapter's cross-metric view adds no new evidence for that claim; its contribution is narrower and more

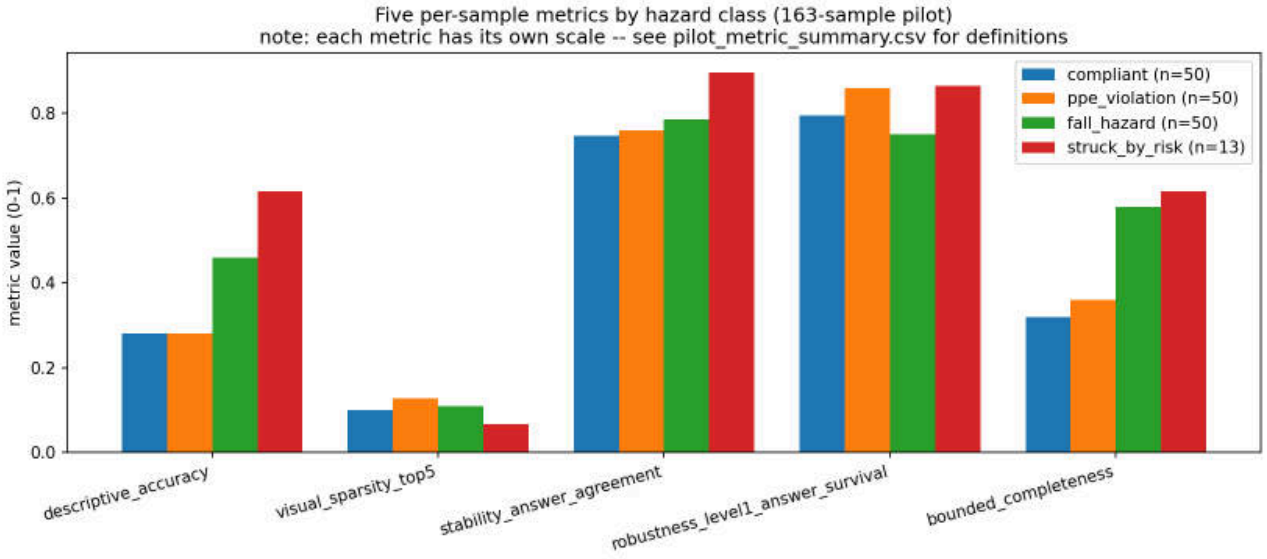


Figure 11.2: The headline rollup chart: five per-sample metrics (descriptive accuracy, visual sparsity, stability, robustness, bounded completeness), each on its own scale, broken down by the pilot's four hazard classes. Efficiency is omitted here because it is a single aggregate runtime figure with no per-class structure to plot.

disciplined — making the consequence of that already-traced mechanism visible in one chart, while flagging plainly that the two bars moving together is not itself a second piece of evidence, so that the eventual paper does not double-count one finding as two.

11.6 Finding 2: visual sparsity runs opposite from descriptive_accuracy/completeness for struck_by_risk

struck_by_risk has the *highest* descriptive_accuracy (61.5%) and the *highest* bounded_completeness (61.5%) of any class — but the *lowest* visual_sparsity_top5 (0.067, against 0.099–0.127 for the other three classes). The class whose top-ranked region is most reliably load-bearing under masking is the same class whose attention is least concentrated by the cross-attention attribution signal. **"Focused attention" and "decision-critical region" are not the same property**, and struck_by_risk is this pilot's clearest case of the two pointing in different directions at once.

The plausible mechanism traces to rule_4's attributed objects. rule_4's proximity check grounds `"excavator"` and `"worker"` — both large structural regions, not small point-like objects in the way a hard hat is. Florence-2's grounding-query attention naturally spreads over a bigger area for a physically bigger object, the same size confound Chapter 6 already documented driving the hard-hat/excavator concentration gap (top-5 mass ratio 0.195 versus 0.079, correlation −0.70 between log box area and concentration). That same spread-out attention pattern lowers struck_by_risk's top-5 mass ratio here. It does not, however, make the masking test any less decisive: masking that whole large excavator-or-worker region still reliably destroys the spatial relationship rule_4's proximity check depends on, which is exactly why descriptive_accuracy and bounded_completeness stay high for this class even as visual_sparsity_top5 falls. This is, at $n = 13$ across a four-class comparison, a real but weak signal — consistent with, and a direct continuation of, the running theme already established in Chapters 6 and 9: sparsity and accuracy/completeness measure genuinely different properties of an explanation

and must be reported separately, never blended into one undifferentiated "explanation quality" score.

11.7 Finding 3: robustness is the flattest metric across classes, and fall_hazard is its worst case — not its best, unlike elsewhere

Robustness Level 1 varies only from 75.0% to 86.5% across the four classes — an 11.5 percentage-point spread, far narrower than the roughly 30–60 point spans the other four per-sample metrics show in Table 11.1. Within that flat metric, **fall_hazard is the single worst class (75.0%)**, which is notable specifically because fall_hazard is the *second-best* class for both descriptive_accuracy (46.0%) and bounded_completeness (58.0%). No other class reverses position this sharply between the two metric groups.

This matches, rather than contradicts, what Chapter 9 already found at the rule level: fall_hazard pools rule_2 (harness/lanyard) and rule_3 (guardrail/edge barrier) samples, and these two sub-rules destabilize under perturbation for **two different reasons**. rule_2 has the highest `worker_lost` fallback-rerouting rate of any rule under occlusion, 19.2% (Chapter 9, Finding 1) — occlusion disrupts Florence-2's whole-scene grounding enough that no worker survives detection, triggering a degenerate fallback branch unrelated to whether the harness itself is actually still visible. rule_3, by contrast, has the highest grid-fallback rate of any rule (Chapter 10, Finding 3: 3 of the 4 total grid-fallback samples in the entire pilot are rule_3), reflecting that "guardrail" is a harder open-vocabulary grounding target for Florence-2 than the other rules' queried objects. Averaging rule_2's and rule_3's results into one `fall_hazard` number here makes the class look moderately fragile under perturbation without revealing either underlying cause — both of which are already documented individually, at the rule level, in their respective chapters' findings. The class-level number in this chapter's table is real, but it is a blend of two distinct failure modes, not evidence of a single fall_hazard-specific robustness weakness.

11.8 Finding 4: efficiency comfortably scales

Summing Chapter 8's `extrapolated_1000_hours` figure across the four inference-bearing pipeline steps it actually logged — baseline inference, descriptive-accuracy mask reruns, attribution, and stability's threefold sampled reruns; robustness and bounded completeness's reruns reuse already-timed call types rather than being separately logged, per Chapter 8 — gives **∼0.50 GPU-hours** for the complete six-metric pipeline extrapolated to $n = 1{,}000$ samples, well under an hour of compute on a single RTX 3070[1]. This is the one metric in the entire pilot where bigger is unambiguously better news for a future full-dataset study: there is no compute bottleneck standing between this 163-sample pilot and a 1,000-sample, or larger, follow-up run of the same six-metric pipeline.

11.9 Standard vs. Non-Standard Choices

Standard: rolling up several already-computed per-metric results into one summary table and one comparison chart is itself standard practice for any multi-metric evaluation framework paper, and mirrors the reporting style of [3], the same six-metric framework this pilot adapts — a single cross-metric figure or table, alongside each metric's own detailed results, is the conventional way such frameworks are presented. Nothing about *producing* that rollup departs from convention; `11_make_figures.py`

[1] `results/efficiency_summary.csv` has exactly four rows (Days 3, 5, 6, 7); an earlier informal note on this finding described "six logged pipeline steps," which this chapter corrects against the primary CSV.

performs no new model calls and no recomputed logic, only `groupby(primary_class).mean()` over columns Chapters 5–10 already wrote and already verified.

Non-standard: the contribution in this chapter is not the rollup itself but two explicit cautions attached to it, neither of which a generic multi-metric rollup would necessarily surface on its own. First, Finding 1's flagging that descriptive_accuracy and bounded_completeness's shared cross-class shape is a near-identity of construction, not two independent confirmations — a caution against double-counting one finding as two simply because it appears in two different metric columns. Second, Finding 2's reading of the sparsity-versus-completeness divergence for struck_by_risk as positive evidence that the six metrics measure genuinely different properties of an explanation, not six redundant proxies for one underlying "explanation quality" score. Both cautions require checking *how* a metric was computed, not only what value it produced, before treating cross-metric agreement or disagreement as meaningful — a discipline a purely numerical rollup does not enforce by itself.

11.10 Implications for the Paper

This chapter's headline table, Table 11.1, is essentially the paper's Results-section centerpiece: it is the one place in this pilot where all six metrics from [3]'s framework, adapted to Florence-2 and the ConstructionSite safety domain, are reported together, on a shared four-class breakdown, in a form a reader can take in in a single table or a single chart rather than assembling it themselves from six separate chapters. Figure 11.2 should be the paper's primary results figure, full width, exactly as presented here; no other single figure in this pilot carries as much of the report's quantitative content at a glance.

Findings 1 and 2, read together, are the strongest argument this chapter contributes to the paper's broader methodological case: **the six metrics are not redundant, and a rollup chapter's job is partly to demonstrate that non-redundancy explicitly, not merely to display the numbers side by side.** Finding 1 shows that two metrics can agree closely across classes for a reason that has nothing to do with independent corroboration — one is partially constructed from the other — and a paper that reported this agreement as "two metrics confirm the same finding" without that caveat would overstate its own evidence. Finding 2 shows the complementary case: two metrics can *disagree* sharply for a class (struck_by_risk) for a reason that is itself informative — a large structural attributed region versus a small point-like one — rather than indicating that one of the two metrics is simply wrong. Both findings should be stated explicitly in the paper's discussion of the six-metric framework's internal validity, since a reviewer's first question about any multi-metric evaluation system is almost always whether its metrics are actually measuring different things or merely restating one signal six ways.

Finding 3 carries forward a caution already raised independently in Chapters 9 and 10: any class- or rule-level aggregate that pools sub-rules with different underlying failure modes (here, rule_2's fallback-rerouting fragility and rule_3's grounding-difficulty fragility, both folded into one `fall_hazard` number) risks masking both causes behind a single moderate-looking statistic. The paper should report sub-rule breakdowns alongside class-level numbers wherever a class, like fall_hazard, pools structurally different rules, rather than treating the class-level number as self-explanatory. Finally, Diagram F (Figure 11.1) should anchor the paper's own architecture or methods figure: it is the only diagram in this pilot that shows the complete eleven-script pipeline in one image, and Day 13's from-scratch reproducibility rerun, later in this report, is, in substance, a validation that this exact pipeline — end to end, all eleven scripts, all five Diagrams A through E composed together — runs correctly outside the original development environment.

Chapter 12

Synthesizing Findings into a Pilot Report (Day 12)

12.1 Motivation

Chapters 1–11 each ran, or rolled up, one stage of the pilot — environment setup, sample selection, baseline inference, region extraction, and the six metrics in turn — and reported its own results against its own evidence. None of those chapters had the job of stepping back and asking what the eleven days look like *as a single body of work*, written for a reader (Professor Mustafa Abdallah) who was not present for any individual day and needs one document rather than eleven. Day 12's task was exactly that: produce a 4–6 page technical memo synthesizing Days 1–11 for the upcoming collaboration meeting — motivation, dataset, model, methods, the six metrics' headline results, cross-cutting findings, limitations, and next steps — ahead of, and separate from, this chapter-by-chapter report itself. The artifact produced is `pilot/report/pilot_report_draft.md` (commit `c2a1f85`).

12.2 The Report's Structure and Editorial Policy

The memo's structure follows the natural order of the pilot itself rather than a generic report template: Motivation → Dataset → Model and the grounding-proxy adaptation → Methods summary (one table mapping each of the six metrics to its method) → Results (one headline table) → Cross-cutting findings → Limitations → Next steps and the specific points where Professor Abdallah's guidance is wanted. This is the same structure this chapter set later expanded, day by day, into Chapters 1–11.

One editorial policy adopted for the memo is worth reporting explicitly as a methodological choice in its own right, not just as a stylistic habit. The memo's own header states it verbatim: **"Every number below traces to a specific CSV or figure produced in Days 1-11 (`pilot/results/`); none are invented or estimated for this writeup."** Every figure quoted in the memo — the 163-sample actual n, the per-rule sensitivity numbers, the six metrics' headline and per-class values, the ∼0.5 GPU-hour efficiency estimate — was checked against the specific results file that produced it before being written down, rather than recalled from memory or rounded for readability. This discipline costs time under a 14-day timebox, where a faster memo could have been drafted from the day-by-day findings narratives alone; it was kept anyway, because a synthesis document that mixes verified and remembered numbers without flagging which is which is exactly the kind of artifact that erodes trust once a single number turns out to be wrong.

12.3 Cross-Cutting Synthesis as Its Own Activity

The memo's most distinct contribution is not any single new measurement — Day 12 ran no new model calls and computed no new metric — but the act of reading the eleven days side by side and noticing

where the *same* mechanism surfaces independently in more than one of them. **Noticing a mechanism recur across independently-run days is a different activity from, and adds information beyond, any single day's own finding**: a day-by-day report tells the reader what happened on each day; a synthesis tells the reader which of those eleven stories are actually one story told from several angles. This is the activity that turns eleven days of isolated results into a coherent picture, and it is why Day 12 exists as a separate step rather than being folded into Day 11's already-completed rollup.

Two recurring mechanisms, each first identified independently within a single day's own analysis and only connected explicitly during this synthesis pass, illustrate the activity:

The fallback-rerouting mechanism (Chapters 5, 9, 10). Masking the worker region does not only remove visual evidence; for samples where the worker box was the only detected box, it also removes the precondition the person-relative check needs to run at all, silently rerouting the proxy to a cruder scene-level fallback whose answer depends on a different signal entirely. Chapter 5 first surfaced this as the explanation for an otherwise paradoxical result (masking *more* regions producing a slightly *lower* flip rate). Chapter 9 found the same mechanism again, independently, as a driver of answer-flips under occlusion. Chapter 10 found it a third time, independently, inflating a small slice of completeness verdicts. No single one of those three chapters claims to have discovered a general property of the pipeline; together, they show one.

The area-based region-ranking confound (Chapters 4, 6, 7, 10). Chapter 4 ranks candidate regions by raw pixel area, by design, with no detection-class signal available to do otherwise. That single design choice then resurfaces as the explanation for the weakest result in three separate, independently-run metric chapters: Chapter 6's sparsity measurements concentrate mechanically for small objects regardless of reasoning quality; Chapter 7's stability IoU is markedly lower for the small hard-hat box than for the large excavator box; and Chapter 10's completeness rate is lower for rule_1 (PPE, small object) than for rule_3 (guardrail, often the largest object in frame). Each of the three metric chapters reported its own number as that metric's own finding; only reading all four chapters together makes clear that one upstream ranking heuristic, not three unrelated weaknesses, is responsible.

12.4 Standard vs. Non-Standard Choices

Standard: producing a structured technical report that synthesizes a multi-stage empirical pilot — motivation, methods, results, limitations, next steps — ahead of a collaboration meeting is itself an entirely ordinary practice, and the memo's section ordering follows that convention closely.

Non-standard, and worth naming explicitly: the strict "every number must trace to a specific file" sourcing discipline described above, maintained even under a 14-day timebox where it would have been easy, and faster, to write the memo from recollection of each day's findings narrative rather than re-opening each day's CSV or figure to confirm the exact value before quoting it. Most technical memos written under this kind of deadline pressure relax that discipline somewhere; this one did not.

12.5 Implications for the Paper

The process by which the cross-cutting findings above were produced matters as much as the findings themselves, for the paper's methodology section. Findings were tracked day by day, inside each day's own chapter, at the time each day's work was done — the fallback-rerouting mechanism was written down as a Day 5 finding before Day 9 or Day 10 existed, and the area-ranking confound was written down as a Day 4 design note before Day 6, Day 7, or Day 10 existed. The cross-cutting connections were drawn only afterward, during this Day 12 synthesis pass, by rereading the already-written day-by-day

findings side by side. No day's findings narrative was revised after the fact to make a later day's discovery look like it had been anticipated. This ordering is itself a methodological safeguard worth stating in the paper: a synthesis that retrofits early findings to match a later, more complete picture risks manufacturing the appearance of a coherent story where the evidence trail does not actually support one. Reporting the two recurring mechanisms above as genuine cross-chapter discoveries, rather than as if they had been the plan from Day 1, is only credible because the day-by-day record this report is built from shows exactly when each piece was actually found.

This also sets the template the paper should follow: a day-by-day (or stage-by-stage) results narrative, written contemporaneously, followed by a separate synthesis pass that explicitly distinguishes single-stage findings from cross-stage patterns, rather than a single flat results section that blends the two. The pilot's own evidence for why that separation matters is the two mechanisms above — neither would have been visible from any one stage's results table alone.

Chapter 13

From-Scratch Reproducibility – and a Real Bug Found by Testing the Process (Day 13)

13.1 Motivation

Chapters 1–12 establish, day by day, that the eleven-script pipeline runs correctly inside the one development environment it was written in. None of those twelve chapters answer a different, and equally necessary, question: does the pipeline run correctly *anywhere else*? A pilot whose results only reproduce inside the original author's already-configured venv, with whatever undocumented state has accumulated there over twelve days, is a weaker artifact than one whose results reproduce from a clean checkout on a machine that has never seen this project before. The plan anticipated this distinction explicitly and set a verification criterion for Day 13 accordingly:

> *"the from-scratch 20-sample rerun completing without manual intervention is the actual test."*

The emphasis in that sentence is deliberate and worth restating plainly: the test is not whether the pipeline *can be made to work* from scratch with some amount of troubleshooting; the test is whether it works from scratch *without* troubleshooting. Any step that required manual intervention – a hand-edited path, a skipped error, a workaround discovered only by reading a traceback – would itself be a finding, because it would mean the documented setup procedure (Chapter 1's `requirements.txt` and install order) is incomplete or wrong for anyone other than the person who already knows its undocumented exceptions.

13.2 Method

The clean-room procedure was built to isolate the question of "does the pipeline reproduce" from every other variable. The exact committed `pilot/` tree at `HEAD` was exported with `git archive HEAD pilot` into a throwaway directory entirely outside the repository (`D:\xai_pilot_repro_check\pilot`), guaranteeing the rerun used only what was actually committed, not whatever uncommitted scratch files happened to be sitting in the working tree. A brand-new virtual environment was created inside that throwaway directory, and `pip install` was run from a clean slate – no packages carried over from the main venv, no shared site-packages.

The rerun used a fresh 20-sample subset rather than the full 163-sample pilot, by deliberate design: `SAMPLES_PER_CLASS` was patched from 50 to 5 in that throwaway copy only (5 samples $\times$ 4 classes $= 20$), never touching the real `pilot/data` or `pilot/results`. A smaller n keeps the check fast – every script still has to run end to end, but on a fifth of the samples Chapter 2 selected for the main pilot – while

still exercising every code path the full run exercises: every script from `01_check_environment.py` through `11_make_figures.py` ran, in order, against this fresh subset, inside the fresh venv, against the genuinely freshly-installed dependency set.

13.3 Result: Full Pipeline Reproduces, Exit 0 on Every Script

Table 13.1 reports the per-script outcome of the rerun. Every step exited 0; nothing was skipped, retried by hand, or patched mid-run.

Step	Exit	Notes
`pytest tests/ -v`	0	61/61 passed, same as the main venv
`01_check_environment.py`	0	GPU confirmed (`cuda:0`, fp16); gated dataset access worked using the existing user-level HF token – no extra login needed, since `huggingface_hub`'s token cache lives at `~/.cache/huggingface`, not inside the venv
`02_select_samples.py`	0	wrote exactly 20 rows, 5/class, no underfilled-class warning
`03_run_baseline_inference.py`	0	20/20
`04_extract_regions.py`	0	20/20, grid-fallback 5.0% (1/20)
`05_descriptive_accuracy.py`	0	20/20 – see the network-blip finding below
`06_visual_sparsity.py`	0	20/20
`07_stability_test.py`	0	20/20
`08_efficiency_test.py`	0	20/20 + 15-sample calibration
`09_robustness_test.py`	0	20/20 + patch-stretch
`10_bounded_completeness.py`	0	20/20
`11_make_figures.py`	0	wrote summary CSV + chart – see the bug-fix finding below

Table 13.1: Per-script outcome of the from-scratch 20-sample reproducibility rerun. All twelve steps (the test suite plus eleven pipeline scripts) exited 0.

No manual intervention was needed at any step. The documented install order – pinned-CUDA `torch` first, then `requirements.txt`, then `pip install -e .` – worked exactly as written. This specific order matters because of a known failure mode it is designed to avoid: installing from `requirements.txt` before the pinned-CUDA `torch` can silently replace an already-installed CUDA build of `torch` with a CPU-only one, since a generic `requirements.txt` entry for `torch` is satisfied by either. The exact-version pin (`torch==2.12.1`) meant this rerun's second install step did not trigger that replacement – verified directly, not assumed: `torch.__version__` was `2.12.1+cu130` and `torch.cuda.is_available()` was `True` both before and after `pip install -r requirements.txt` ran.

13.4 Two Real Things This Rerun Surfaced

A clean-room rerun that simply confirmed every script exits 0 would already satisfy the plan's verification criterion. This rerun did more than that: it surfaced two real things, one of which is this chapter's centerpiece finding.

1. A genuine bug in `11_make_figures.py`, found and fixed. Before this rerun, the chart-generation code in `11_make_figures.py` hardcoded its legend and title text: the per-class legend labels were built from a literal string check (`"13" if cls == "struck_by_risk" else "50"`), and the chart's title hardcoded the literal phrase `"163-sample pilot"`. Both values are correct for the main 163-sample pilot Chapters 5–11 already ran and reported. Neither value is read from data – they are typed directly into the plotting code as literal strings. That made the bug invisible for twelve days, because every run of `11_make_figures.py` up to this point *was* a run over the 163-sample data, so the hardcoded numbers happened to be right every single time they were checked. This 20-sample rerun is the first time the script ever ran over data the hardcoded numbers do not describe, and it would have produced a chart silently mislabeled `"163-sample pilot"` with `n=50` legends sitting next to bars actually computed from five samples each per class – a chart that looks complete and correct and is wrong in a way no amount of staring at the bars themselves would reveal.

The fix replaces both hardcoded values with values read directly from the data already loaded earlier in the same script: the per-class legend counts now come from `da["primary_class"].value_counts()` (where `da` is the already-loaded `descriptive_accuracy.csv` DataFrame), and the chart title's sample count now comes from `len(da)`, rather than a literal `"163"`. Concretely, the bar-construction line changed from

```
label=f"{cls} (n={'13' if cls == 'struck_by_risk' else '50'})"
```

to

```
label=f"{cls} (n={class_counts.get(cls, 0)})"
```

and the title's hardcoded `"163-sample pilot"` fragment became an f-string interpolating `len(da)`. The fix touches only chart-label construction; no computation, aggregation, or metric logic changes. After fixing it inside the throwaway copy, `11_make_figures.py` was re-run against the real 163-sample data to confirm the fix did not silently change anything in the main pilot's own output: the regenerated `pilot_metric_summary.csv` was byte-identical in its numbers to the pre-fix version, and the regenerated `results/figures/summary/metric_summary_by_class.png` was visually re-confirmed correct – still `n=50/50/50/13` and `"163-sample pilot"`, because that is still the true sample count for the main pilot. The fix was then applied to the actual tracked `pilot/scripts/11_make_figures.py` in this repository, not left isolated inside the throwaway copy.

This is the chapter's central finding, and it is worth stating its significance plainly rather than folding it into a list of minor observations. The bug had been sitting in committed, tested code for the entire span of Chapter 11's rollup, undetected, because every test that exercised `11_make_figures.py` – including `pytest tests/`'s 61 passing tests – exercised it against the one sample size for which the hardcoded numbers happen to be correct. A clean-room reproducibility rerun at a *different* sample size is precisely the kind of check that exposes this class of bug, because it is the first execution path that disagrees with the hardcoded assumption. No code review of `11_make_figures.py` alone, however careful, would have caught this without either reading every literal against its source data line by line or actually running the script against different data and watching the label diverge from reality.

2. A transient network error during `05_descriptive_accuracy.py`'s dataset streaming, auto-recovered. An `IncompleteRead`/`ProtocolError` occurred while streaming the test-split parquet file from the Hub mid-run. This was not a code bug, and the rerun did not need to intervene to fix it: `huggingface_hub`'s built-in retry logic (three retries, exponential backoff) caught the error, retried automatically, and succeeded, and the script went on to exit 0 as recorded in Table 13.1. It is flagged here, separately from Finding 1, because it is a real source of pipeline flakiness worth being aware of for any future full-dataset run: at larger scale, or under less reliable network conditions than this rerun happened to encounter, the same error class recurring more than three times in a row would exhaust the built-in retry budget and fail the run outright. The appropriate response, if this is seen recurring in a future run, is a `-retries`/timeout configuration tweak on the streaming call, not a code fix to this pilot's own logic – and not urgent at this pilot's scale, where it occurred once and resolved itself.

13.5 20-Sample vs. 163-Sample Numbers

This rerun's purpose was reproducing the *pipeline*, not reproducing the *exact numbers* at roughly eight times fewer samples, and the comparison in Table 13.2 should be read with that framing in mind throughout.

Metric	163-sample (real)	20-sample (repro check)
descriptive_accuracy	36.2%	30.0%
visual_sparsity_top5	0.108	0.092
stability_answer_agreement	77.5%	93.3%
robustness_level1_answer_survival	80.7%	83.8%
bounded_completeness	43.6%	35.0%
efficiency @ n=1,000 (GPU-hours)	∼0.50	∼0.54

Table 13.2: The six headline metrics, real 163-sample pilot versus the from-scratch 20-sample reproducibility check.

Four of the six metrics land close to the real pilot's numbers. `stability_answer_agreement` is the clear outlier – 93.3% at $n = 20$ against 77.5% at $n = 163$, a 15.8 percentage-point gap far wider than any of the other five metrics' gaps. This is expected sampling noise at this size, not a discrepancy that calls the pipeline's correctness into question: twenty samples spread across four classes is a small enough draw that one or two samples landing on the "stable" side of Chapter 7's threefold-rerun agreement check swings the class mean substantially, in a way that a 50-samples-per-class draw mostly averages out. Treating a 15.8-point swing at $n = 20$ as evidence of a pipeline bug would be a misread of what a from-scratch check at a deliberately reduced sample size is and is not capable of confirming.

The more meaningful reproducibility signal sits one level below the raw percentages: the rule-level ordering that Chapters 6, 7, and 10 already explained mechanistically – excavator-class (struck_by_risk, rule_4) objects scoring lowest on sparsity because Florence-2's attention spreads over their larger attributed regions, and rule_4 scoring highest on stability and robustness because its proximity check is comparatively robust to perturbation – holds in both the 163-sample and the 20-sample runs. Two independently-run draws of the same pipeline, at different sample sizes, agreeing on *which class is*

mechanistically strongest and weakest for each metric is stronger evidence that the pipeline measures something real than the two draws agreeing on the exact percentage would have been. Exact-number agreement at an 8x size difference would, if anything, have been mildly suspicious; ordering agreement is exactly what a correctly-functioning pipeline run at a smaller n should produce.

Cleanup. The throwaway venv and the 20-sample data and results it produced (the `xai_pilot_repro_check` directory, created outside the repository) were deleted after this check completed. They were scratch space for this verification only, never a tracked artifact, and no file from that throwaway directory is committed to this repository.

13.6 Standard vs. Non-Standard Choices

Standard: a clean-room reproducibility check – fresh virtual environment, fresh export of the committed tree, documented install steps followed exactly as written, no shortcuts taken because the person running the check happens to already know where the bodies are buried – is itself increasingly expected practice for empirical ML papers, formalized in artifact-evaluation and reproducibility checklists at venues such as NeurIPS and ACL. Running such a check at all, ahead of a paper submission rather than only in response to a reviewer's request, is the unsurprising half of this chapter.

Non-standard, and the actual contribution: explicitly treating the reproducibility check itself as a **test of the pipeline's robustness** – something that can fail and reveal a real defect – rather than as a formality that exists only to produce a checkbox for the camera-ready submission. Most reproducibility checks are run expecting, and finding, nothing; this one was run expecting nothing and found a real bug, precisely because it exercised a code path (a non-163 sample size) the original twelve days of development never had occasion to exercise. The second non-standard choice compounds the first: deliberately choosing a reduced $n = 20$ rather than rerunning the full 163-sample pilot from scratch was not merely a speed optimization. A reduced n that still walks every script in order is what made this check fast enough to run inside the Day 13 timebox while still being the kind of run most likely to expose an assumption – like a hardcoded sample count – that only holds at one specific n. Rerunning the full 163-sample pilot from scratch would have taken longer and would not have caught Finding 1 at all, because at $n = 163$ the hardcoded numbers are correct by construction.

13.7 Implications for the Paper

This chapter doubles as the paper's Reproducibility section, and its central exhibit is the bug-found-by-testing-the-process narrative above, not the step-by-step exit-code table that precedes it. A table of eleven exit-0 results is necessary evidence but, on its own, makes a comparatively thin methodological argument – it tells a reader that the pipeline ran, not that running it from scratch was a meaningfully different activity from running it again in the original environment. Finding 1 supplies that argument directly: it demonstrates a defect that was real, already committed, already covered by a passing 61/61 test suite, and invisible to every check this pilot had run for twelve days, surfaced specifically and only because the reproducibility check ran the pipeline against an input it had never seen before.

The concrete case this makes for the paper's methodology section is that reproducibility checks belong in the pilot phase, run alongside the metrics work rather than deferred to camera-ready preparation. Had this check been deferred – treated as a final formality performed only once a paper draft already existed, immediately before submission – the same bug would very likely still have been caught eventually, most plausibly by a reviewer or a third party attempting to reproduce the paper's own results on different data, at a point in the process where fixing it costs considerably more than it cost here: a

few lines changed in an already-open script, verified in minutes by rerunning one already-fast script against the original data. Running the check during the pilot, while the codebase is small enough that a single rerun script can exercise the entire pipeline end to end in minutes, converts reproducibility verification from a release-gating risk into a routine development check with a fast feedback loop – the same argument, in substance, that motivates running a unit test suite continuously rather than only before a release.

This chapter's evidence also closes a loop Chapter 11 opened. Diagram F (Chapter 11's capstone architecture figure) asserted that the eleven-script pipeline, assembled from Diagrams A through E, is the complete pilot pipeline end to end. This chapter is, in substance, the empirical validation of that assertion: Diagram F's eleven scripts, run in the exact order the diagram specifies, on a machine that had never seen this codebase before, produced exit 0 at every node and a defect-corrected chart at the final one. A diagram describing an architecture and a from-scratch rerun confirming that architecture actually executes as drawn are different kinds of evidence, and the paper should present both rather than treating the diagram alone as sufficient documentation of how the pipeline runs.

Chapter 14

Packaging the Pilot for a Research Collaboration Meeting (Day 14)

14.1 Motivation

Chapters 1–13 document thirteen days of building, testing, and verifying a six-metric evaluation pipeline against a vision-language model. None of those chapters' artifacts are written for the specific occasion this pilot was always aiming toward: a meeting with Professor Mustafa Abdallah, the collaborator whose six-metric framework this pilot adapts, who was not present for any of the preceding thirteen days and needs a small set of focused materials rather than a thirteen-chapter report. Day 14's task, per the two-week pilot plan, was to finalize that material: a one-page pitch, a curated set of sample visualizations, a task list for the full study framed as decisions rather than to-dos, and a five-minute spoken explanation covering what was tested, what worked, what failed, and where Professor Abdallah's expertise is specifically needed. This chapter is not a fourteenth unit of evaluation work; it is the closing act of packaging thirteen days of it for the person the work was done for.

14.2 Artifacts Produced

Four documents were produced on Day 14, each serving a different moment in the meeting rather than duplicating the same content four times. `report/one_pager_summary.md` is the single page meant to be read before or during the meeting: the question, the headline answer, the six metrics' headline numbers, the findings that matter more than those numbers, and the points where guidance is wanted. `report/talking_points.md` is the five-minute spoken structure built to walk through that same material out loud — the question (30 seconds), what was tested (one minute), what worked (1.5 minutes), what failed or is more interesting than the headline numbers (1.5 minutes), and where guidance is wanted (30 seconds) — speaking notes rather than a script. `report/key_visualizations.md` curates six images out of the 60-plus saved across this pilot's thirteen days, each selected as the clearest single-image evidence of a specific finding rather than for visual appeal. `report/full_study_task_list.md` converts six open items into six explicit decisions, each naming the chapter whose evidence motivates it, so the meeting's task-list discussion starts from pilot evidence rather than a generic future-work list.

Every number in all four documents traces to a results file or a chapter already in this report; none was invented or estimated for the meeting package. This is the same sourcing discipline Chapter 12's pilot-report memo held under its own 14-day timebox, inherited here rather than relaxed for what is, in volume, the smallest of this pilot's written artifacts.

14.3 The Headline Answer

Chapter 1 opened this report by establishing that the pilot was executable at all; underneath that operational question sat the substantive one this entire pilot exists to answer: **does Professor Abdallah's six-metric XAI evaluation framework — Descriptive Accuracy, Sparsity, Stability, Efficiency, Robustness, and Completeness, built and validated on tabular intrusion-detection DNNs — transfer to a vision-language model making visual judgments?** Thirteen chapters of evidence now stand behind a direct answer.

Yes, with two real, quantified adaptations — not zero changes. All six metrics ran end-to-end against Florence-2-base-ft (231M parameters, a single RTX 3070) on 163 construction-safety images sampled from `LouisChen15/ConstructionSite`'s 3,004-image test split. Neither adaptation was optional or cosmetic; each was a response to a concrete failure the framework's own measurements surfaced, not a precaution taken in advance of trouble.

The first adaptation is Chapter 3's grounding-proxy reframing. Florence-2 has no native free-form visual-question-answering token, so each safety rule had to be reframed as a grounding query against the model's actual task vocabulary (`<OPEN_VOCABULARY_DETECTION>`) rather than posed as a literal yes/no question. This adaptation sits at the model-interface level, not the metric level: none of the six metrics had to be redefined to accommodate it, but every one of them depends on the resulting proxy answer being a meaningful judgment in the first place.

The second adaptation is the person-relative redesign, also from Chapter 3, found only after the first version of the grounding proxy was tested and found wanting: a scene-level existence check achieved 3.0% sensitivity, a near-constant "compliant" classifier. Redesigning the proxy to check coverage per detected worker, mirroring the dataset's own per-worker labeling semantics, produced a **9× sensitivity gain (27.0%)**. This is the more consequential of the two adaptations: Descriptive Accuracy and Bounded Completeness, the framework's own metrics, measure whether an *explanation* is faithful to a judgment — they do not, by themselves, measure whether the underlying judgment-generation method is discriminating at all. A near-constant classifier would have left those metrics almost nothing real to test, and the framework's own machinery would not have flagged that on its own; it took a separate sensitivity/specificity check, run before any explanation method was built on the proxy, to catch it.

With both adaptations in place, all six metrics produced genuine, interpretable signal: 36.2% descriptive accuracy, 0.108 top-5 visual sparsity, 77.5% stability, 80.7% Level-1 robustness, 43.6% bounded completeness, and a confirmed-trivial ∼0.5 GPU-hours per 1,000 samples for the full pipeline. None of the six metrics had to be abandoned or silently redefined into something the framework's original authors would not recognize. The honest reading of these thirteen chapters is that the framework's *structure* transferred intact across the gap from tabular intrusion-detection features to image-and-text VLM inputs; what did not transfer for free was the assumption, implicit in applying any of the six metrics, that the system being evaluated already produces a discriminating judgment to explain. That assumption needed to be checked and, on first attempt, repaired — exactly the kind of adaptation a feasibility pilot exists to surface before a larger study is built on an unverified premise.

14.4 Curated Visualizations

`report/key_visualizations.md` curates six images out of the more than sixty saved across Chapters 3–11, selected because each is the clearest single-image evidence for one specific finding, not for being the most visually striking option available. Each is identified here by the chapter that produced it and the finding it illustrates; none is re-embedded in this chapter, since each already appears, in full, at the

point in this report where it was first introduced and explained.

1. **The headline rollup** (`summary/metric_summary_by_class.png`, first shown as Figure 11.2). Chapter 11's capstone chart: all five per-sample metrics, broken down by hazard class, on their own native scales. This is the single chart that the rest of the curated set exists to explain — everything else on this list accounts for some feature of why this chart looks the way it does.
2. **The grounding-proxy adaptation, working** (`baseline/0000007_rule_1.png`, first shown as Figure 3.2). Chapter 3's worked example of the redesigned person-relative proxy in action: a hard-hat query correctly localized on a worker's head, the underlying mechanism behind the 3.0%-to-27.0% sensitivity gain that is this pilot's second headline adaptation.
3. **Descriptive accuracy on a real sample** (`descriptive_accuracy/0000037_rule_1_flip.png`). The clearest single-image illustration, among Chapter 5's ten saved flip overlays, of masking the top-ranked region changing the model's answer — descriptive accuracy behaving exactly as the metric is designed to detect.
4. **Sparsity, focused versus scattered.** `sparsity/0000023_rule_1_hard_hat.png` and `sparsity/0000079_rule_3_guardrail.png`, first shown together as Figure 6.2. Chapter 6's side-by-side pair makes the chapter's size-confound finding visible without requiring the correlation number: a small, sharp hotspot on a distant hard hat against a broadly scattered heatmap for a frame-spanning guardrail, two physically different targets rather than two differently-explained judgments.
5. **A stable answer hiding a drifted explanation** (`robustness/verify/0000034_before.png` and `robustness/verify/0000034_after_occlude.png`, first shown together as Figure 9.3). The single best image in this pilot for Chapter 9's centerpiece finding: the same final answer before and after occlusion, while the model's evidence box quietly relocates to something unrelated (`object_box_iou=0.005`) — the case that motivates reporting answer-level and explanation-level robustness as two never-blended numbers.
6. **Compute is not the obstacle** (`efficiency/per_sample_cost.png`, first shown as Figure 8.1). The per-sample timing breakdown behind Chapter 8's ∼0.5-GPU-hours-at-n=1,000 figure, the evidence that efficiency is the one metric in this pilot whose answer is unqualified good news for scaling up.

14.5 Six Decisions for Professor Abdallah

`report/full_study_task_list.md` converts six items this pilot deliberately scoped down or left open into six explicit decisions, each framed as something requiring Professor Abdallah's judgment before it is worth committing engineering time to, and each tied to the specific chapter whose evidence motivates it. None of the six is something the pilot attempted and failed at; each is a fork in the road the pilot reached and stopped at, by design, rather than a defect to be silently fixed.

1. Attribution method (Chapter 6). Is decoder→encoder cross-attention an acceptable stand-in for LRP or DeepLIFT in the framework's own terms, or does a transformer-native attribution method need to be added before the framework's attribution component can be considered fully ported? Chapter 6 verified directly that classic attention rollout's single-modality assumption does not hold for Florence-2's fused image+text encoder, and substituted cross-attention only after that verification, but the substitution's acceptability as formal evidence for the framework's purposes is a judgment call this pilot is not positioned to make on its own.

2. Statistical rigor at small n, especially `struck_by_risk` (Chapters 2, 11). Chapter 2 found that the entire 3,004-image test split contains only 13 `struck_by_risk` examples, capping that class's sample size well below the other three classes' 50 each; Chapter 11 flagged every `struck_by_risk` number in its headline table as a directional signal rather than a stable estimate for exactly this reason. What minimum n per class should be required before a metric is reported as a stable estimate, and what test or interval should accompany each headline number going forward, is a decision this pilot defers rather than resolves.

3. Region-ranking heuristic (Chapters 4, 10). Should candidate regions be ranked by whether they match the rule's queried object class before falling back to raw area, removing the worker-versus-PPE-object confound that Chapter 4 flagged as a foreseen limitation at the moment `standardize_regions` was designed and that Chapter 10 later measured directly: masking the worker flips the answer in only 37.5% of cases against 49.4% for the actual safety object, an 11.9 percentage-point gap traced to one specific, replaceable ranking function. This is the single highest-leverage, most fixable item on this list, and fixing it plausibly improves three metrics' weakest results simultaneously for PPE classes — but it is listed as a decision, not an authorized fix, because it changes the regions every later masking-based chapter in this pilot depends on.

4. Full-dataset scaling versus the `struck_by_risk` class imbalance (Chapter 8). Is scaling to the full 3,004-image test split the right next step, or is correcting the 13-example `struck_by_risk` shortage a higher-value use of the same engineering effort? Chapter 8 confirmed that compute is not the deciding factor either way: the full six-metric pipeline costs roughly half a GPU-hour at n=1,000 on a single consumer-grade RTX 3070, so this decision should turn on what scaling or rebalancing strategy best serves the eventual paper's evidentiary needs, not on what a single RTX 3070 can afford to run.

5. A real Level-3 robustness study (Chapter 9). What scope and budget should a properly powered adversarial-patch study receive? Chapter 9's 20-image stretch test had only 4 candidates for which a flip to "compliant" was even a meaningful outcome, and those four cases showed no clean placement-to-flip correlation — a well-placed patch failed to fool the model, a badly-placed one succeeded. That result is a plausibility signal worth a properly powered follow-up, explicitly not a result strong enough to support a claim about Florence-2's vulnerability to adversarial patches on its own.

6. `rule_2`'s prompt-phrasing ambiguity (Chapter 9, Finding 3). Should `rule_2`'s two phrasings, "safety harness" and "fall-protection lanyard," be treated as two distinct physical concepts rather than wording variants of one concept, before `rule_2`'s robustness numbers are trusted in a larger study? Chapter 9 found a 50.0% Level-2 answer-change rate for `rule_2` against 10.2% for `rule_3` under the same rewording test, and traced the gap to the two `rule_2` phrasings plausibly grounding two different objects on the same scene, not to any instability in how Florence-2 handles a reworded question.

14.6 Standard vs. Non-Standard Choices

Standard: assembling a stakeholder-facing summary package — a one-pager, talking points, and a curated set of visuals — at the close of a feasibility pilot, ahead of a collaboration meeting, is itself an entirely ordinary practice. Nothing about producing four short documents to support a five-minute spoken explanation departs from how any applied research pilot would normally close out before reporting to a collaborator or sponsor.

Non-standard, and the choice worth naming explicitly: framing every open item in the full study task list as an explicit decision that names the specific pilot evidence behind it, rather than as a

generic "future work" list compiled after the fact. A generic future-work list would have let each of the six items in Section 14.5 read as an unexamined gap; instead, each is presented as a fork already reached, with the chapter and finding that produced it cited directly, so that Professor Abdallah's judgment is brought to bear on a decision with evidence already attached rather than on an open-ended request for direction. This mirrors, at the level of the whole pilot's closing package, the same discipline Chapter 12's pilot-report memo adopted at the level of a single document: every number traces to a specific source, and every open question traces to a specific chapter's finding, rather than either being supplied from memory or general impression.

14.7 Closing Synthesis

Chapter 1 began this report by removing one assumption — that the pilot was executable at all — so that every later day's plan would rest on verified ground rather than on faith. Thirteen chapters later, the report closes by answering the question that operational verification was always in service of: **does Professor Abdallah's six-metric XAI evaluation framework, built and validated on tabular intrusion-detection DNNs, transfer to a vision-language model making visual judgments on construction-safety imagery?** Section 14.3 gave that answer directly — yes, with two real, quantified adaptations, not zero changes — and it is worth restating here, at the report's close, why that answer is the right level of confidence to report rather than either a more triumphant or a more hedged one.

It would overstate the evidence to claim the framework transferred unmodified: Chapter 3's person-relative redesign was a 9× change in the underlying proxy's discriminative power, found only because the adoption process included a direct sensitivity check the six metrics themselves do not perform. It would equally understate the evidence to treat that adaptation as a sign the framework does not generalize: every one of the six metrics, once given a discriminating judgment to explain, ran to completion and produced real, interpretable, traceable signal — Chapter 5's non-monotonicity anomaly, Chapter 6's size confound, Chapter 9's answer-level-versus-explanation-level split, and Chapter 10's area-ranking finding are exactly the kind of findings a working evaluation framework produces when pointed at a real system, not symptoms of a framework failing to fit its new target. The thirteen chapters behind this answer were written contemporaneously, day by day, as Chapter 12 recorded, and the cross-cutting patterns connecting them — the fallback-rerouting mechanism recurring across Chapters 5, 9, and 10; the area-ranking confound recurring across Chapters 4, 6, 7, and 10 — were connected only afterward, on a rereading of an already-written record, not retrofitted to manufacture a tidier story than the evidence supports. That is the discipline this closing synthesis relies on: the six-metric framework transfers to a vision-language model, and the report's own day-by-day record is the reason that claim can be made with the confidence it is made here.

14.8 Implications for the Paper

This chapter is the natural home for the eventual paper's Conclusion and Future Work section, and the two pieces of this chapter map onto that section almost directly. Section 14.3's headline answer — transfer succeeds, with two quantified, named adaptations — is the Conclusion: a single paragraph the paper's abstract and introduction can both be built around, with Chapters 1–13 supplying the supporting evidence in full behind it. Section 14.5's six decisions are the Future Work section's candidate bullets almost without rewriting: each already names the specific chapter and finding that motivates it, which is precisely the structure a Future Work section should have — specific, evidenced, and prioritized, rather than a closing paragraph of generic caveats.

Two of the six decisions deserve to be highlighted as priorities if the paper's Future Work section needs to rank rather than simply list them. The region-ranking heuristic (decision 3) is the single highest-leverage fix identified anywhere in this pilot, named as a foreseen limitation as early as Chapter 4 and measured directly by Chapter 10: it requires no new model calls, only a re-ranking of boxes Florence-2 has already returned, and Chapter 10 already supplies a falsifiable prediction for what a successful fix should produce. The attribution-method question (decision 1) is the most consequential for how the paper positions its own methodological contribution: whether decoder→encoder cross-attention is accepted as sufficient evidence for a fused-modality vision-language model, or whether the framework requires a transformer-native method before its attribution component can be considered fully ported, determines how strongly the paper can claim the full framework, rather than five of its six components, has been validated on a VLM. Both decisions, and the four alongside them, are now in Professor Abdallah's hands with this pilot's evidence already attached — which is the entire purpose this closing chapter, and the meeting it was written for, was built to serve.

Bibliography

[1] Samira Abnar and Willem Zuidema. Quantifying attention flow in transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4190–4197, 2020. doi: 10.18653/v1/2020.acl-main.385.

[2] Osvaldo Arreche and Mustafa Abdallah. A comparative analysis of DNN-based white-box explainable AI methods in network security. *EURASIP Journal on Information Security*, 2025(1):16, 2025. doi: 10.1186/s13635-025-00201-x.

[3] Osvaldo Arreche, Tanish R. Guntur, Jack W. Roberts, and Mustafa Abdallah. E-xai: Evaluating black-box explainable ai frameworks for network intrusion detection. *IEEE Access*, 12:23954–23988, 2024. doi: 10.1109/ACCESS.2024.3365140.

[4] Sebastian Bach, Alexander Binder, Grégoire Montavon, Frederick Klauschen, Klaus-Robert Müller, and Wojciech Samek. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLOS ONE*, 10(7):e0130140, 2015. doi: 10.1371/journal.pone.0130140.

[5] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, and Orion Reblitz-Richardson. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896*, 2020.

[6] LouisChen15. ConstructionSite: A construction-site safety vision-language dataset. Hugging Face Hub dataset, `https://huggingface.co/datasets/LouisChen15/ConstructionSite`, 2024. Accessed June 2026. No associated peer-reviewed publication was found at time of writing.

[7] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3145–3153, 2017.

[8] Bin Xiao, Haiping Wu, Weijian Xu, Xiyang Dai, Houdong Hu, Yumao Lu, Michael Zeng, Ce Liu, and Lu Yuan. Florence-2: Advancing a unified representation for a variety of vision tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.